

An Initial Introduction to Cooperative Multi-Agent Reinforcement Learning

Christopher Amato, Northeastern University

May 22, 2025

Contents

1	Introduction	5
1.1	Overview	5
1.2	The cooperative MARL problem: The Dec-POMDP	6
1.3	Background on (single-agent) reinforcement learning	9
1.3.1	Value-based methods	10
1.3.2	Policy gradient methods	12
2	Centralized training and execution (CTE)	17
2.1	CTE overview	17
2.2	Centralized models	18
2.3	Centralized solutions	19
2.4	Improving scalability	20
3	Decentralized training and execution (DTE)	21
3.1	DTE overview	21
3.2	Decentralized, value-based methods	22
3.2.1	Independent Q-learning (IQL)	22
3.2.2	Improving the performance of IQL	24
3.2.3	Deep extensions, issues, and fixes	26
3.3	Decentralized policy gradient methods	30
3.3.1	Decentralized REINFORCE	30
3.3.2	Independent actor critic (IAC)	31
3.3.3	Other decentralized policy gradient methods	32
3.4	Other topics	33
4	Centralized training for decentralized execution (CTDE)	35
4.1	CTDE overview	35
4.2	Value function factorization methods	36
4.2.1	VDN	36
4.2.2	QMIX	38
4.2.3	Weighted QMIX	40
4.2.4	QTRAN	41
4.2.5	QPLEX	43
4.2.6	The use of state in factorization methods	44
4.2.7	Other value function factorization methods	44
4.3	Centralized critic methods	45

4.3.1	Preliminaries	45
4.3.2	A basic centralized critic approach	45
4.3.3	MADDPG	49
4.3.4	COMA	50
4.3.5	MAPPO	50
4.3.6	State-based critics	52
4.3.7	Choosing different types of decentralized and centralized critics	53
4.3.8	Methods that combine policy gradient and value factorization	54
4.3.9	Other centralized critic methods	54
4.4	Other forms of CTDE	54
4.4.1	Adding centralized information to decentralized methods	54
4.4.2	Decentralizing centralized solutions	55
5	Conclusion	57
5.1	Topics not discussed	57
5.2	Evaluation domains	57
5.3	Applications	57
5.4	Future topics	58

Chapter 1

Introduction

1.1 Overview

Multi-agent reinforcement learning (MARL) has exploded in popularity in recent years. While numerous approaches have been developed, they can be broadly categorized into three main types: centralized training and execution (CTE), centralized training for decentralized execution (CTDE), and decentralized training and execution (DTE).

CTE methods assume centralization during training and execution (e.g., with fast, free, and perfect communication) and have the most information during execution. That is, the actions of each agent can depend on the information from all agents. As a result, a simple form of CTE can be achieved by using a single-agent RL method with centralized action and observation spaces (maintaining a centralized action-observation history for the partially observable case). CTE methods can potentially outperform the decentralized execution methods (since they allow centralized control) but are less scalable as the (centralized) action and observation spaces scale exponentially with the number of agents. CTE is typically only used in the cooperative MARL case since centralized control implies coordination on what actions will be selected by each agent.

CTDE methods are the most common, as they leverage centralized information during training while enabling decentralized execution—using only information available to that agent during execution. CTDE is the only paradigm that requires a separate training phase where any available information (e.g., other agent policies, underlying states) can be used. As a result, they can be more scalable than CTE methods, do not require communication during execution, and can often perform well. CTDE fits most naturally with the cooperative case, but may also apply in competitive or mixed settings depending on what information is assumed to be observed.

Decentralized training and execution methods make the fewest assumptions and are often simple to implement. In fact, any single-agent RL method can be used for DTE by just letting each agent learn separately. Naturally, these approaches have their respective advantages and disadvantages, which will be discussed. It is worth noting that DTE is required if no centralized training phase is available (e.g., through a centralized simulator), requiring all agents to learn during online interactions without prior coordination. DTE methods can be applied in cooperative, competitive, or mixed cases.

MARL methods can be further categorized into value-based and policy gradient methods. Value-based methods (e.g., Q-learning) learn a value function and then choose actions based on those values. Policy gradient methods learn an explicit policy representation and attempt to im-

prove the policy in the direction of the gradient. Both classes of methods are widely used in MARL.

This text is an introduction to cooperative MARL—MARL in which all agents share a single, joint reward. It is meant to explain the setting, basic concepts, and common methods for the CTE, CTDE, and DTE settings. It does not cover all work in cooperative MARL as the area is quite extensive. I have included work that I believe is important for understanding the main concepts in the area and apologize to those that I have omitted.

I will first give a brief description of the cooperative MARL problem in the form of the Dec-POMDP as well as a short background on the (single-agent) reinforcement learning methods that are relevant to this text. Then, I will discuss the centralized, decentralized, and CTDE settings and algorithms. Note that all the presented methods are model-free (i.e., they do not learn a model of the dynamics or observations) since the vast majority of cooperative MARL methods are model-free.

For the centralized case, I will discuss the setting as well as ways of modeling the CTE problem, simple applications of single-agent methods as well as some more scalable methods that exploit the multi-agent structure.

For the decentralized case, I will give an overview of the DTE setting and then I will discuss the relevant value-based and policy gradient methods. I'll start with value-based DTE methods such as independent Q-learning and its extensions and then I will also discuss the extension to the deep case with DQN, the additional complications this causes, and methods that have been developed to (attempt to) address these issues. Next, I will discuss policy gradient DTE methods starting with independent REINFORCE [Peshkin et al., 2000] (i.e., vanilla policy gradient), and then extending to the actor-critic case and deep variants (such as independent PPO [de Witt et al., 2020, Yu et al., 2022]). Finally, I will discuss some general topics related to DTE and future directions.

Finally, I will present an overview of CTDE and the two main classes of CTDE methods: value function factorization methods and centralized critic actor-critic methods. Value function factorization methods include the well-known VDN [Sunehag et al., 2017], QMIX [Rashid et al., 2018], and QPLEX [Wang et al., 2021a] approaches, while centralized critic methods include MADDPG [Lowe et al., 2017], COMA [Foerster et al., 2018b], and MAPPO [Yu et al., 2022]. Lastly, I discuss other forms of CTDE such as adding centralized information to decentralized (i.e., independent) learners (such as parameter sharing) and decentralizing centralized solutions.

A detailed treatment of reinforcement learning (in the single-agent setting) is not presented in this text. Anyone interested in RL should read the book by Sutton and Barto [2018]. Similarly, for a broader overview of MARL, the recent book by Albrecht, Christianos and Schäfer is recommended [Albrecht et al., 2024].

1.2 The cooperative MARL problem: The Dec-POMDP

The cooperative multi-agent reinforcement learning (MARL) problem can be represented as a decentralized partially observable Markov decision process (Dec-POMDP) [Oliehoek and Amato, 2016, Bernstein et al., 2002]. Dec-POMDPs generalize POMDPs [Kaelbling et al., 1998] (and MDPs [Puterman, 1994]) to the multi-agent, decentralized setting. They are also a subclass of partially observable stochastic games (POSGs) where all the agents have the same reward (i.e., a cooperative POSG). As depicted in Figure 1.1, multiple agents operate under uncertainty based on partial views of the world, with execution unfolding over time. At each step, every agent

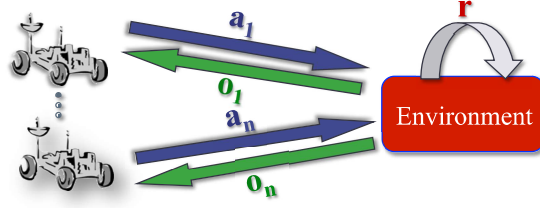


Figure 1.1: A depiction of cooperative MARL—a Dec-POMDP.

chooses an action (in parallel) based purely on locally observable information, resulting in each agent obtaining an observation and the team obtaining a joint reward. The shared reward function makes the problem cooperative, but their local views mean that execution is decentralized.

Formally, a Dec-POMDP is defined by tuple $\langle \mathbb{I}, \mathbb{S}, \{\mathbb{A}_i\}, T, R, \{\mathbb{O}_i\}, O, \mathcal{H}, \gamma \rangle$. For simplicity, I define the finite version but it easily extends to the continuous case:

- $\mathbb{I}$ is a finite set of agents of size $|\mathbb{I}| = n$;
- $\mathbb{S}$ is a finite set of states with designated initial state distribution b_0 ;¹
- $\mathbb{A}_i$ is a finite set of actions for each agent i with $\mathbb{A} = \times_i \mathbb{A}_i$ the set of joint actions;
- T is a state transition probability function, $T : \mathbb{S} \times \mathbb{A} \times \mathbb{S} \rightarrow [0, 1]$, that specifies the probability of transitioning from state $s \in \mathbb{S}$ to $s' \in \mathbb{S}$ when the actions $\mathbf{a} \in \mathbb{A}$ are taken by the agents (i.e., $T(s, \mathbf{a}, s') = \Pr(s' | \mathbf{a}, s)$);
- R is a reward function: $R : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{R}$, the immediate reward for being in state $s \in \mathbb{S}$ and taking the actions $\mathbf{a} \in \mathbb{A}$;
- $\mathbb{O}_i$ is a finite set of observations for each agent, i , with $\mathbb{O} = \times_i \mathbb{O}_i$ the set of joint observations;
- O is an observation probability function: $O : \mathbb{O} \times \mathbb{A} \times \mathbb{S} \rightarrow [0, 1]$, the probability of seeing observations $\mathbf{o} \in \mathbb{O}$ given actions $\mathbf{a} \in \mathbb{A}$ were taken and resulting in state $s' \in \mathbb{S}$ (i.e., $O(\mathbf{a}, s', \mathbf{o}) = \Pr(\mathbf{o} | \mathbf{a}, s')$);
- $\mathcal{H}$ is the number of steps until termination, called the horizon;
- and $\gamma \in [0, 1]$ is the discount factor.

I also include both the horizon and discount in the definition but, typically, only one is used. When $\mathcal{H}$ is finite, γ can be set to 1 and when $\mathcal{H} = \infty$, $\gamma \in [0, 1)$.²

A solution to a Dec-POMDP is a *joint policy*, denoted π —a set of policies, one for each agent, each of which is denoted π_i . Because the state is not directly observed, it is typically beneficial for each agent to remember a history of its observations (and potentially actions). Then, a *local policy*, π_i , for an agent is a mapping from local action-observation histories to actions or probability

¹Some papers refer to other information such as joint observations or histories as ‘state.’ For clarity, I will only use this term to refer to the true underlying state.

²The episodic case [Sutton and Barto, 2018] is typically indefinite-horizon with termination to a set of terminal states with probability 1 and the infinite-horizon case is sometimes called ‘continuing.’

distributions over actions. A *local deterministic policy* for agent i , $\pi_i(h_i) = a_i$, maps $\mathbb{H}_i \rightarrow \mathbb{A}_i$, where $\mathbb{H}_i$ is the set of *local observation histories*, $h_i = \{a_{i,0}, o_{i,0}, \dots, a_{i,t-1}, o_{i,t-1}\}^3$, by agent i up to the current time step, t . Note that histories have implicit time steps due to their length which I do not include in the notation (i.e., I always assume a history starts on the first time step and the last time step is defined by the number of action-observation pairs). We can denote the *joint histories* for all agents at a given time step as $\mathbf{h} = \langle h_1, \dots, h_n \rangle$. A *joint deterministic policy* is denoted $\pi(\mathbf{h}) = \mathbf{a} = \langle \pi_1(h_1), \dots, \pi_n(h_n) \rangle = \langle a_1, \dots, a_n \rangle$. A *stochastic local policy* for agent i is $\pi_i(a_i|h_i)$, representing the probability of choosing action a_i in history h_i . A *joint stochastic policy* is denoted $\pi(\mathbf{a}|\mathbf{h}) = \prod_{i \in \mathbb{I}} \pi_i(a_i|h_i)$. Because one policy is generated for each agent and these policies depend only on local observations, they operate in a decentralized manner.

Many researchers just use observation histories (without including actions), which is sufficient for deterministic policies but may not be for stochastic policies [Oliehoek and Amato, 2016]. Deterministic policies will be used in the value-based methods while stochastic policies will be used in the policy gradient methods. There always exists an optimal deterministic joint policy in Dec-POMDPs [Oliehoek and Amato, 2016], but stochastic (or continuous) policies are needed for policy gradient methods.

The value of a joint policy, π , at joint history $\mathbf{h}$ can be defined for the case of discrete states and observations as

$$\mathbf{V}^\pi(\mathbf{h}) = \sum_s P(s|\mathbf{h}, b_0) \left[R(s, \mathbf{a}) + \gamma \sum_{s'} P(s'|\mathbf{a}, s) \sum_{\mathbf{o}} P(\mathbf{o}|\mathbf{a}, s') \mathbf{V}^\pi(\mathbf{hao}) \right] \Big|_{\mathbf{a}=\pi(\mathbf{h})} \quad (1.1)$$

where $P(s|\mathbf{h}, b_0)$ is the probability of state s after observing joint history $\mathbf{h}$ starting from state distribution b_0 and $\mathbf{a} = \pi(\mathbf{h})$ is the joint action taken at the joint history. Also, $\mathbf{hao}$ represents $\mathbf{h}'$, the joint history after taking joint action $\mathbf{a}$ in joint history $\mathbf{h}$ and observing joint observation $\mathbf{o}$. In the RL context, algorithms do not iterate over states and observations to explicitly calculate this expectation but approximate it through sampling. For the finite-horizon case, $\mathbf{V}^\pi(\mathbf{h}) = 0$ when the length of $\mathbf{h}$ equals the horizon $\mathcal{H}$, showing the value function includes the time step from the history.

We will often want to evaluate policies starting from the beginning—starting at the initial state distribution before any action or observation. Starting from this initial, null history I denote the value of a policy as $\mathbf{V}^\pi(\mathbf{h}_0)$. An *optimal joint policy* beginning at $\mathbf{h}_0$ is $\arg\max_\pi \mathbf{V}^\pi(\mathbf{h}_0)$, where the $\arg\max$ here denotes enumeration over decentralized policies. The optimal joint policy is then the set of local policies for each agent that provides the highest value, which is denoted $\mathbf{V}^*$.

Reinforcement learning methods often use history-action values, $\mathbf{Q}(\mathbf{h}, \mathbf{a})$, rather than just history values $\mathbf{V}(\mathbf{h})$. $\mathbf{Q}^\pi(\mathbf{h}, \mathbf{a})$ is the value of choosing joint action $\mathbf{a}$ at joint history $\mathbf{h}$ and then continuing with policy π ,

$$\mathbf{Q}^\pi(\mathbf{h}, \mathbf{a}) = \sum_s P(s|\mathbf{h}, b_0) \left[R(\mathbf{a}, s) + \gamma \sum_{s'} P(s'|\mathbf{a}, s) \sum_{\mathbf{o}} P(\mathbf{o}|\mathbf{a}, s') \mathbf{Q}^\pi(\mathbf{hao}, \mathbf{a}') \Big|_{\mathbf{a}'=\pi(\mathbf{hao})} \right], \quad (1.2)$$

while $\mathbf{Q}^*(\mathbf{h}, \mathbf{a})$ is the value of choosing action $\mathbf{a}$ at history $\mathbf{h}$ and then continuing with the optimal policy, π^* .

Policies that depend only on a single observation It is somewhat popular to define policies that depend only on a single observation rather than the whole observation history. That is, $\pi_i :$

³Sometimes, $o_{i,0}$ is also included as an observation generated by the initial state distribution

$\mathbb{O}_i \rightarrow \mathbb{A}_i$, rather than $\mathbb{H}_i \rightarrow \mathbb{A}_i$. This type of policy is often referred to as *reactive* or *Markov* since it just depends on (or reacts from) the past observation. These reactive policies are typically not desirable since they can be arbitrarily worse than policies that consider history information [Murphy, 2000] but can perform well or even be optimal in simpler subclasses of Dec-POMDPs [Goldman and Zilberstein, 2004]. In general, reactive policies reduce the complexity of finding a solution (since many fewer policies need to be considered) but only perform well in limited cases such as when the problem is not really partially observable or when the observation history isn't helpful for decision-making.

The fully observable case It is worth noting that if the cooperative problem is fully observable, it becomes much simpler. Specifically, a (decentralized) multi-agent MDP (MMDP) can be defined by the tuple $\langle \mathbb{I}, \mathbb{S}, \{\mathbb{A}_i\}, T, R, \mathcal{H}, \gamma \rangle$ [Boutilier, 1996]. Each agent can observe the full state in this case. In the CTDE context, solving an MMDP can be done by standard MDP methods where the action space is the joint action space, $\mathbb{A} = \times_i \mathbb{A}_i$. The resulting policy, $\mathbb{S} \rightarrow \mathbb{A}$, can be trivially decentralized as $\mathbb{S} \rightarrow \mathbb{A}_i$ for each agent i . A number of methods have been developed for learning in MMDPs (and other multi-agent models) [Buşoniu et al., 2008]. More details about solving MMDPs (for decentralized and centralized control) are given in Section 2.

Finding solutions in Dec-POMDPs is much more challenging. This can even be seen by the (finite-horizon) complexity for finite problems—optimally solving an MDP is P-complete (polynomial in the size of the system) [Papadimitriou and Tsitsiklis, 1987, Littman et al., 1995] and optimally solving a POMDP is PSPACE (essentially exponential in the actions and observations) [Papadimitriou and Tsitsiklis, 1987], while optimally solving a Dec-POMDP is NEXP-complete (essentially doubly exponential) [Bernstein et al., 2002]. This can be intuitively understood by considering the simplest possible solution method for each class of problems. For an MDP, you could search all policies mapping states to (joint) actions: $|\mathbb{A}|^{|\mathbb{S}|}$ of them. In a POMDP, policies consider histories and for a given horizon $\mathcal{H}$, there are $|\mathbb{O}|^{\mathcal{H}}$ possible observation histories. Up to horizon $\mathcal{H}$, there are then $\sum_{t=0}^{\mathcal{H}-1} (|\mathbb{O}|)^t = \frac{|\mathbb{O}|^{\mathcal{H}} - 1}{|\mathbb{O}| - 1}$ possible observation histories. Deterministic policies map from these observation histories to actions, giving $|\mathbb{A}|^{\frac{|\mathbb{O}|^{\mathcal{H}} - 1}{|\mathbb{O}| - 1}}$ possible policies. In the Dec-POMDP case, each agent would have $|\mathbb{A}_i|^{\frac{|\mathbb{O}_i|^{\mathcal{H}} - 1}{|\mathbb{O}_i| - 1}}$ possible policies, so the possible number of possible joint policies would be $|\mathbb{A}_i|^{\frac{|\mathbb{O}_i|^{\mathcal{H}} - 1}{|\mathbb{O}_i| - 1} |\mathbb{I}|}$ if all agents have the same number of actions and observations as agent i . Of course, state-of-the-art methods are more sophisticated than this and the complexity is for the worst case. Most real-world problems are intractable to solve optimally anyway. Regardless, these numbers make it clear that searching the joint space of history-based policies is enormous and the resulting partially observable problem is much more challenging than the fully observable case.

1.3 Background on (single-agent) reinforcement learning

While I don't present a full introduction to reinforcement learning (which can be found in other texts [Sutton and Barto, 2018]), I will discuss some of the key methods that MARL methods build on. I break the discussion up into value-based methods (e.g., Q-learning) and policy gradient methods, discussing the fully observable case as well as the extension to the partially observable setting.

1.3.1 Value-based methods

Value-based methods learn a value function and then derive the agent’s policy from the value function. While Q-learning [Watkins and Dayan, 1992] is the most famous such method, the class of value-based methods is much larger. Nevertheless, all the value-based MARL methods that I discuss are based on some form of Q-learning. Therefore, I focus on these Q-learning-based methods below.

Q-learning [Watkins and Dayan, 1992] is a simple and widely used method for value-based RL. In the single-agent, fully-observable case (i.e., an MDP), Q-learning maintains a table of Q-values for each state-action pair. These Q-values are initialized (e.g., randomly) and then the agent takes steps in the environment given the current state as input. That is, given the current state, s , the agent chooses an action, a , and then observes the next state s' and receives the immediate reward r . With this information, the Q-value for s and a is updated with:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

The quality $r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ is called the TD-error and will be denoted as δ (resulting in $Q(s, a) \leftarrow Q(s, a) + \alpha\delta$). Q-learning has been shown to converge (in single-agent, fully-observable environments) with minor assumptions (such as proper exploration and reduction of the learning rate) [Watkins and Dayan, 1992, Melo, 2001].

Q-learning can be extended to the partially observable case (i.e., a POMDP [Kaelbling et al., 1998]) by using histories rather than states.⁴ As noted above, in partially observable environments, the state is not directly observable so agents may need to remember information from the past. It turns out that a POMDP can be transformed into a history-MDP where the ‘states’ of the history-MDP are the histories of the POMDP [Nguyen, 2021]. Q-learning (and any fully-observable RL method) can then be extended to consider histories instead of states. The agent chooses actions, a , based on the current action-observation history, h , observes the resulting observation, o and receives an immediate reward, r . The Q-learning update becomes:

$$Q(h, a) \leftarrow Q(h, a) + \alpha \left[r + \gamma \max_{a'} Q(h', a') - Q(h, a) \right]$$

where $h' = hao$.

Deep Q-networks (DQN) [Mnih et al., 2015] is an extension of Q-learning to include a neural net as a function approximator. Since DQN was designed for the fully observable (MDP) case, it learns $Q_\theta(s, a)$, parameterized with θ (i.e., θ represents the parameters of the neural network), by minimizing the loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{\langle s, a, r, s' \rangle \sim \mathcal{D}} \left[(y - Q_\theta(s, a))^2 \right], \text{ where } y = r + \gamma \max_{a'} Q_{\theta^-}(s', a') \quad (1.3)$$

which is just the squared TD error—the difference between the current estimated value, $Q_\theta(s, a)$, and the new value gotten from adding the newly seen reward to the previous Q-estimate at the next state, $Q_{\theta^-}(s', a')$. Because learning neural networks can be unstable, a separate target action-value function Q_{θ^-} and an experience replay buffer $\mathcal{D}$ [Lin, 1992] are implemented to stabilize

⁴*Beliefs*, or distributions over states, are often used in the model-based or planning cases. However, they require a transition and observation model to calculate so are not typically used in RL.

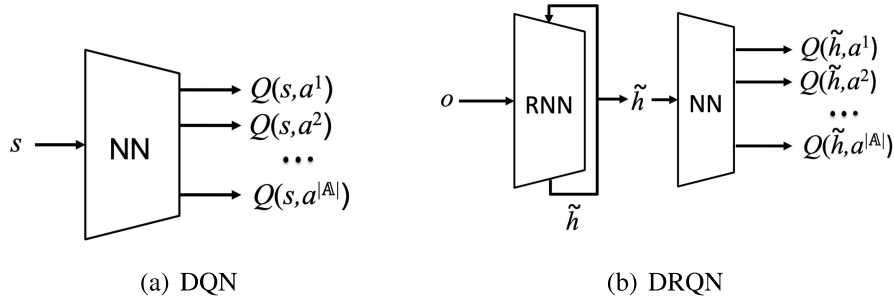


Figure 1.2: DQN and DRQN network diagrams.

learning. The target network is an older version of the Q-estimator that is updated periodically with $\langle s, a, r, s' \rangle$ sequences stored in the experience replay buffer and single $\langle s, a, r, s' \rangle$ tuples are i.i.d. sampled for updates. As shown in Figure 1.2(a), the neural network (NN) outputs values for all actions ($a \in \mathbb{A}$) to make maximizing over all states possible with a single forward pass (rather than iterating through the actions).

Deep recurrent Q-networks (DRQN) [Hausknecht and Stone, 2015] extends DQN to handle partial observability, where some number of recurrent layers (e.g., LSTM [Hochreiter and Schmidhuber, 1997]) are included to maintain an internal hidden state which is an abstraction of the history (as shown as $\tilde{h}$ in Figure 1.2(b)). Because the problem is partially observable, o, a, r, o' sequences are stored in the replay buffer during execution. The update equation is very similar to that of DQN:

$$\mathcal{L}(\theta) = \mathbb{E}_{\langle h, a, r, o \rangle \sim \mathcal{D}} \left[(y - Q_{\theta}(h, a))^2 \right], \text{ where } y = r + \gamma \max_{a'} Q_{\theta^-}(h', a') \quad (1.4)$$

but since the recurrent neural network (RNN) is used, the internal state of the RNN can be thought of as a history representation. RNNs are trained sequentially, updating the internal state at each time step. As a result, the figure shows only o as the input but this assumes the internal state $\tilde{h}$ has already been updated using the history up to this point h^{t-1} . Therefore, rather than just sampling single updates (o, a, r, o') i.i.d, like in DQN, histories are sampled from the buffer. The internal state can be updated incrementally and the Q-values learned starting from the first time step and going until the end of the history (i.e., the horizon or episode). Technically, a whole history (e.g., episode) should be sampled from the replay buffer to train the recurrent network but it is common to use a fixed history length (e.g., sample o, a, o', r sequences of length 10). Also, as mentioned above, many implementations just use observation histories rather than full action-observation histories. Note that I will still write h rather than $\tilde{h}$ in the equations (e.g., $Q_{\theta}(h, a)$ above) to be more general. The resulting equations are not restricted to recurrent models (e.g., non-recurrent representation such as the full history or a Transformer-based representation [Esslinger et al., 2022, Ni et al., 2023]). With the popularity of DRQN, it has become common to add recurrent layers to standard (fully observable) deep reinforcement learning methods when solving partially observable problems.

Pseudocode for a version of DRQN is shown in Algorithm 1. The algorithm uses a learning rate, α , some form of exploration, which in this case is ϵ -greedy, and Q is estimated using an RNN parameterized by θ along with using a target network, θ^- , and a replay buffer, $\mathcal{D}$. Episodes (either a fixed set or until convergence) are generated by interacting with the environment and these are added to the replay buffer. This buffer typically has a fixed size and new data replaces old data

when the buffer is full. Episodes are then be sampled from the replay buffer (a single episode is sampled here but it can also be a minibatch of episodes). In order to have the correct internal state, RNNs are trained sequentially. Training is shown here from the beginning of the episode until the end, calculating the internal state and the loss at each step and updating θ using gradient descent. The target network θ^- is updated periodically (every C episodes in the code) by copying the parameters from θ .

The code is written from the finite-horizon perspective but it can be adjusted to the episodic or indefinite-horizon case by replacing the horizon with terminal states or the infinite-horizon case by removing the loop over episodes. The pseudocode, for this approach and throughout the text, is written to be simple and highlight the main algorithmic components. There are many implementation tricks that are not included.

Algorithm 1 DRQN (finite-horizon*)

```

1: set  $\alpha$ ,  $\epsilon$ , and  $C$  (learning rate, exploration, and target update frequency)
2: Initialize network parameters  $\theta$  and  $\theta^-$  for  $Q_\theta$  and  $Q_\theta^-$ 
3:  $\mathcal{D} \leftarrow \emptyset$ 
4:  $e \leftarrow 1$  {episode index}
5: for all episodes do
6:    $h \leftarrow \emptyset$  {initial history is empty}
7:   for  $t = 1$  to  $\mathcal{H}$  do
8:     Choose  $a$  at  $h$  from  $Q^\theta(h, \cdot)$  with exploration (e.g.,  $\epsilon$ -greedy)
9:     See reward  $r$ , observation
10:    append  $a, o, r$  to  $\mathcal{D}^e$ 
11:     $h \leftarrow hao$  {update RNN state of the network}
12:  end for
13:  sample an episode from  $\mathcal{D}$ 
14:  for  $t = 1$  to  $\mathcal{H}$  do
15:     $h \leftarrow \emptyset$ 
16:     $a, o, r \leftarrow \mathcal{D}^e(t)$ 
17:     $h' \leftarrow hao$ 
18:     $y = r + \gamma \max_{a'} Q^{\theta^-}(h', a')$ 
19:    Perform gradient descent on parameters  $\theta$  with learning rate  $\alpha$  and loss:  $(y - Q^\theta(h, a))^2$ 
20:     $h \leftarrow h'$ 
21:  end for
22:  if  $e \bmod C = 0$  then
23:     $\theta^- \leftarrow \theta$ 
24:  end if
25:   $e \leftarrow e + 1$ 
26: end for
27: return  $Q$ 

```

1.3.2 Policy gradient methods

Policy gradient methods explicitly represent the agent's policy and seek to improve it by using a gradient. The policy is parameterized (e.g., using a neural network) and the parameters can be

updated using gradient ascent to maximize performance (or gradient descent to minimize loss). For example, a network based policy that for a stochastic policy is given in Figure 1.3(a). Performance is typically denoted as J and I will denote the policy parameters (e.g., weights in the neural network) with ψ . For instance, performance is often measured for the policy (which I denote as π^ψ) from the initial state, so $J(\psi) = V^{\pi^\psi}(s_0)$.

The *policy gradient theorem* provides a way to more easily calculate the gradient of the performance metric with respect to the policy parameters by breaking it up into the on-policy state distribution using π^ψ , $\mu(s)$, the Q-value using π^ψ , and the gradient of the policy:

$$\nabla J(\psi) = \sum_s \mu(s) \sum_a Q^{\pi^\psi}(s, a) \nabla \pi^\psi(a|s)$$

where [Sutton and Barto, 2018]. Reinforcement learning algorithms approximate this gradient through sampling.

REINFORCE [Williams, 1992] is one of the simplest policy gradient methods. REINFORCE, like many other policy gradient methods, is an on-policy method so it samples states by following the current policy π^ψ . As a result, $\mu(s)$ is approximated. REINFORCE uses Monte Carlo estimates for the policy value but these will also equal $Q^{\pi^\psi}(s, a)$ in expectation. Assuming a differentiable policy parameterization and sampling an action gives the following update (removing the ψ superscript from π for simplicity of notation):

$$\psi \leftarrow \psi + \alpha \gamma^t G \nabla \log \pi(a|s)$$

where G is the Monte Carlo return starting at the current time step (e.g., for the finite-horizon case: $G = \sum_{k=t}^{\mathcal{H}-1} \gamma^{k-t} r_k$).

Adding a baseline Because Monte Carlo estimates are often high variance, a baseline is often used to reduce the variance. As long as the baseline doesn't depend on the action, it won't change the gradient. Since the baseline can depend on the state, I will denote it as $b(s)$ but a common baseline is a learned value estimate $\hat{V}(s)$. REINFORCE with a baseline then uses the following update:

$$\psi \leftarrow \psi + \alpha \gamma^t (G - b(s)) \nabla \log \pi(a|s).$$

Actor-critic methods combine the TD-learning of value-based methods with policy gradient methods. That is, instead of using Monte Carlo estimate, a TD-based method (e.g., Sarsa or Q-learning) is used to learn a critic, which evaluates the policy. The policy is referred to as the actor, resulting in the actor-critic name. There are many different actor-critic methods (e.g., PPO [Schulman et al., 2017], soft actor critic [Haarnoja et al., 2018]) but a simple actor-critic update using a value estimate, $\hat{V}$, is:

$$\psi \leftarrow \psi + \alpha \gamma^t (r + \hat{V}(s') - \hat{V}(s)) \nabla \log \pi(a|s).$$

Using the δ notation from above, this update can also be written as $\psi \leftarrow \psi + \alpha \gamma^t \delta \nabla \log \pi(a|s)$. Actor-critic methods (and policy gradient methods more generally) are widely used and are variants of the ideas above.

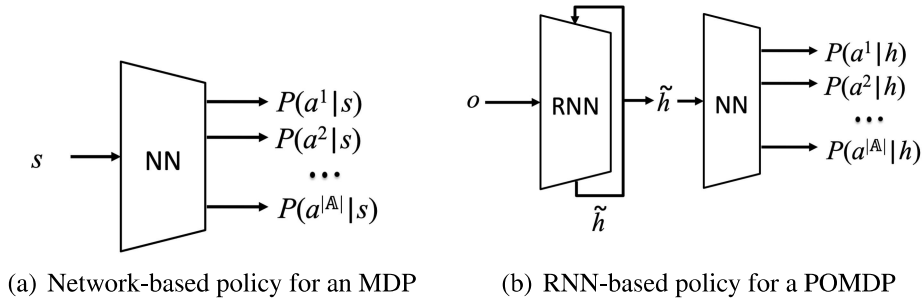


Figure 1.3: State and history-based policies using neural networks.

Partial observability Like the value-based case, we can replace the states of the MDP with histories from the POMDP to extend policy gradient methods to the partially observable case. The policy (and the value function in the actor-critic case) can be represented using neural net such as the example of a stochastic RNN-based policy is given in Figure 1.3(b). Like DRQN [Hausknecht and Stone, 2015], it is common to use recurrent layers (or other methods) to learn history representations. Any of the fully observable methods can be extended but a simple extension of the the actor-critic case can be:

$$\psi \leftarrow \psi + \alpha \gamma^t (r + \hat{V}(h') - \hat{V}(h)) \nabla \log \pi(a|h)$$

where $h' = hao$.

Pseudocode for a simple version of partially observable (advantage) actor-critic is shown in Algorithm 2. This approach is often referred to as advantage actor-critic since it uses the advantage $A(h, a) = Q(h, a) - V(h)$, representing the difference between the Q-value for a given action and the V-value at that history. In the algorithm, $A(h, a)$ isn't explicitly stored since the TD error is used as an approximation for the advantage— $\hat{Q}(h, a)$ is approximated with $r_t + \gamma \hat{V}(h_{t+1})$, which is true in expectation. δ_t can be thought of as a sample of $\hat{Q}(h_t, a_t) - \hat{V}(h_t) = \hat{A}(h_t, a_t)$. Otherwise, actor and critic models are initialized and then data is generated from episodes. On-policy updates are for the value function and the actor and critic are updated as discussed above.

Algorithm 2 Advantage Actor-Critic (A2C) (finite-horizon)

Require: Actor model $\pi(a|h)$, parameterized by ψ **Require:** Critic model $\hat{V}(h)$, parameterized by θ

```

1: Initialize  $\alpha$ , and  $\beta$  learning rates for actor and critic)
2: for all episodes do
3:    $h_0 \leftarrow \emptyset$  { Empty initial history }
4:   for  $t = 0$  to  $\mathcal{H} - 1$  do
5:     Choose  $a_t$  at  $h_t$  from  $\pi(a|h_t)$ 
6:     See reward  $r_t$  and observation  $o_t$ 
7:      $h_{t+1} \leftarrow h_t a_t o_t$  { Append new action and obs to previous history }
8:     Compute value TD error:  $\delta_t \leftarrow r_t + \gamma \hat{V}(h_{t+1}) - \hat{V}(h_t)$ 
9:     Compute actor gradient estimate:  $\gamma^t \delta_t \nabla \log \pi(a_t|h_t)$ 
10:    Update actor parameters  $\psi$  using gradient estimate (e.g.,  $\psi \leftarrow \psi + \alpha \gamma^t \delta_t \nabla \log \pi(a_t|h_t)$ )
11:    Compute critic gradient estimate:  $\delta_t \nabla \hat{V}_i(h_t)$ 
12:    Update critic parameters  $\theta$  using gradient estimate (e.g.,  $\theta \leftarrow \theta + \beta \gamma \delta_t \nabla \hat{V}(h_t)$ )
13:  end for
14: end for
```

Chapter 2

Centralized training and execution (CTE)

2.1 CTE overview

Centralized execution doesn't fit with the Dec-POMDP definition (since policies need to be decentralized!) but discussing the centralized models is useful and there is some notable work in this area. This section begins with assumptions and then discuss the fully observable and the partially observable centralized models, simple centralized solutions, followed by complex approaches leveraging the multi-agent problem structure.

In the CTE case, specifically for the multi-agent POMDP (which is given in detail below), at each step of the problem we assume that:

- a centralized controller chooses actions for each agent, $\mathbf{a}$, based on the current histories for all agents, $\mathbf{h}$ (which are maintained and updated),
- each agent takes the chosen actions $\mathbf{a} = \langle a_1, \dots, a_n \rangle$,
- the centralized controller observes the resulting observations $\mathbf{o} = \langle o_1, \dots, o_n \rangle$
- the (centralized) algorithm observes the current agent histories, $\mathbf{h}$, (or this could be stored from the previous steps) the actions taken by the agents $\mathbf{a}$, the resulting observations, $\mathbf{a}$, and the joint reward r .

I have broken the CTE case into the agents, a centralized controller, and the solution algorithm. In this case, the agents and the controller can be the same since the problem can be considered a single-agent problem using the joint actions, observations, and histories (as discussed below). Nevertheless, I separated them to keep multi-agent execution while emphasizing centralized control. The algorithm is useful to separate from the agent since it has different information (such as rewards) and seeks to optimize the behavior. Agents in any MDP-based model do not condition their policies on rewards so the agents can always operate in environments even without rewards. The rewards are used by the solution algorithm (RL or planning) to evaluate different policies. This separation of algorithm from agent is particularly relevant in the CTDE case where the algorithm has access to centralized information while the agents do not.

2.2 Centralized models

The multi-agent MDP (MMDP) is the fully observable version of the Dec-POMDP, as mentioned in Section 1.2. Unfortunately, the MMDP model doesn't specify whether the control is done in a centralized or decentralized manner. There is a separate model called the Dec-MDP, which assumes the combined observations uniquely determine the state, but agents still only observe their part (e.g., a state feature s_i) [Oliehoek and Amato, 2016]. Similarly, the multi-agent POMDP (MPOMDP) model is used for cases when all agents share their observations at each time step (e.g., see the same observations or send their observations to each other) [Oliehoek and Amato, 2016]. But, again, the MPOMDP model doesn't specify whether control is centralized or decentralized. Therefore, I will use the MMDP and MPOMDP names but be explicit about what type of policy is used.¹

The MMDP can be defined with the tuple $\langle \mathbb{I}, \mathbb{S}, \{\mathbb{A}_i\}, T, R, \mathcal{H}, \gamma \rangle$:

- $\mathbb{I}$ is a finite set of agents of size $|\mathbb{I}| = n$;
- $\mathbb{S}$ is a finite set of states with designated initial state s_0 (or state distribution b_0);
- $\mathbb{A}_i$ is a finite set of actions for each agent i with $\mathbb{A} = \times_i \mathbb{A}_i$ the set of joint actions;
- T is a state transition probability function, $T : \mathbb{S} \times \mathbb{A} \times \mathbb{S} \rightarrow [0, 1]$, that specifies the probability of transitioning from state $s \in \mathbb{S}$ to $s' \in \mathbb{S}$ when the actions $\mathbf{a} \in \mathbb{A}$ are taken by the agents (i.e., $T(s, \mathbf{a}, s') = \Pr(s' | \mathbf{a}, s)$);
- R is a reward function: $R : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{R}$, the immediate reward for being in state $s \in \mathbb{S}$ and taking the actions $\mathbf{a} \in \mathbb{A}$;
- $\mathcal{H}$ is the number of steps until termination, called the horizon;
- and $\gamma \in [0, 1]$ is the discount factor.

Like in Section 1.2, I am including both the horizon and discount in these definitions but, typically, only one is used. Centralized policies in MMDPs map from states to joint actions, $\pi_{MMDP} : \mathbb{S} \rightarrow \mathbb{A}$.

The multi-agent POMDP (MPOMDP) can be defined with the tuple $\langle \mathbb{I}, \mathbb{S}, \{\mathbb{A}_i\}, T, R, \{\mathbb{O}_i\}, O, \mathcal{H}, \gamma \rangle$:

- $\mathbb{I}$ is a finite set of agents of size $|\mathbb{I}| = n$;
- $\mathbb{S}$ is a finite set of states with designated initial state distribution b_0 ;
- $\mathbb{A}_i$ is a finite set of actions for each agent i with $\mathbb{A} = \times_i \mathbb{A}_i$ the set of joint actions;
- T is a state transition probability function, $T : \mathbb{S} \times \mathbb{A} \times \mathbb{S} \rightarrow [0, 1]$, that specifies the probability of transitioning from state $s \in \mathbb{S}$ to $s' \in \mathbb{S}$ when the actions $\mathbf{a} \in \mathbb{A}$ are taken by the agents (i.e., $T(s, \mathbf{a}, s') = \Pr(s' | \mathbf{a}, s)$);

¹I propose to call the centralized control versions the Cent-MMDP and Cent-MPOMDP and the decentralized control versions the Dec-MMDP and the Dec-MPOMDP but this is not standard in the literature.

- R is a reward function: $R : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{R}$, the immediate reward for being in state $s \in \mathbb{S}$ and taking the actions $\mathbf{a} \in \mathbb{A}$;
- $\mathbb{O}_i$ is a finite set of observations for each agent, i , with $\mathbb{O} = \times_i \mathbb{O}_i$ the set of joint observations;
- O is an observation probability function: $O : \mathbb{O} \times \mathbb{A} \times \mathbb{S} \rightarrow [0, 1]$, the probability of seeing observations $\mathbf{o} \in \mathbb{O}$ given actions $\mathbf{a} \in \mathbb{A}$ were taken and resulting in state $s' \in \mathbb{S}$ (i.e., $O(\mathbf{a}, s', \mathbf{o}) = \Pr(\mathbf{o}|\mathbf{a}, s')$);
- $\mathcal{H}$ is the number of steps until termination, called the horizon;
- and $\gamma \in [0, 1]$ is the discount factor.

The MPOMDP definition looks the same as the one for Dec-POMDP but the key difference is the policy used. Dec-POMDPs require a joint policy, $\pi_{Dec-POMDP}$, which is a set of local policies $\pi_{i, Dec-POMDP} : \mathbb{H}_i \rightarrow \mathbb{A}_i$, but in a (centralized) MPOMDP the policies map from *joint histories to joint actions*, $\pi_{MPOMDP} : \mathbb{H} \rightarrow \mathbb{A}$.² As a result, the (centralized) MPOMDP is really just a factored POMDP with the action sets represented as $\mathbb{A} = \times_i \mathbb{A}_i$ and the observation sets represented as $\mathbb{O} = \times_i \mathbb{O}_i$. Similarly, the (centralized) MMDP is really just a factored MDP with action sets $\mathbb{A} = \times_i \mathbb{A}_i$. As a result, standard MDP and POMDP methods can be applied to MMDPs and MPOMDPs³ but such methods may not be scalable due to the potentially large action (and observation) sets. Therefore, scalability has been the focus of CTE MARL methods.

2.3 Centralized solutions

To solve the MMDP, any (fully observable) single-agent RL method can be applied. For instance, the Q-learning update for an MMDP becomes:

$$Q(s, \mathbf{a}) \leftarrow Q(s, \mathbf{a}) + \alpha \left[r + \gamma \max_{\mathbf{a}'} Q(s', \mathbf{a}') - Q(s, \mathbf{a}) \right]$$

where $\mathbf{a} = \langle a_1, \dots, a_n \rangle \in \mathbb{A} = \times_i \mathbb{A}_i$. A centralized solution would be $\pi_{MMDP} : \mathbb{S} \rightarrow \mathbb{A}$ while a decentralized solution would be $\pi_{i, DecentralizedMMDP} : \mathbb{S} \rightarrow \mathbb{A}_i$ for each agent i . For the Q-learning case, the solution is determined by $\pi_{MMDP}(s) = \arg\max_{\mathbf{a}} Q(s, \mathbf{a})$. Once the centralized solution is determined, a decentralized solution is obtained using $\pi_{i, DecentralizedMMDP} = \pi_{MMDP}(s, i)$ where i is the agent index. For example, one possible centralized policy for two agents at state s^{42} could be $\pi_{MMDP}(s^{42}) = \mathbf{a}^7 = \langle a_1^1, a_2^7 \rangle$ while the corresponding decentralized policy would be $\pi_{DecentralizedMMDP} = \langle \pi_1, \pi_2 \rangle$ where $\pi_{1, DecentralizedMMDP}(s^{42}) = a_1^1$ and $\pi_{2, DecentralizedMMDP}(s^{42}) = a_2^7$ (where I would typically drop the agent subscript in the action since it is already in the policy). Because the policy is factored in terms of the agents, there is an action associated with each agent. Other algorithms can be applied by similarly replacing the actions MDP with the joint actions of the MMDP. Note that we are currently assuming that we can first solve the MMDP in a centralized manner. If it must be solved in a decentralized manner,

²Decentralized control in an MPOMDP would result in policies $\mathbb{H} \rightarrow \mathbb{A}_i$.

³As mentioned in Section 1.2 decentralization is trivial once a centralized solution is found for an MMDP and this is also true for an MPOMDP because in both cases the agents observe the same information.

additional coordination is needed to ensure the same policy is learned as mentioned in Section 3 and in other literature [Buşoniu et al., 2008, Littman, 2001].

Solving a MPOMDP can be done in a similar manner. We can transform the MPOMDP into a POMDP where the history, h , becomes the joint history, $\mathbf{h}$ and the actions, a become the joint actions, $\mathbf{a}$. For instance, the Q-learning update for an MPOMDP becomes:

$$Q(\mathbf{h}, \mathbf{a}) \leftarrow Q(\mathbf{h}, \mathbf{a}) + \alpha \left[r + \gamma \max_{\mathbf{a}'} Q(\mathbf{h}', \mathbf{a}') - Q(\mathbf{h}, \mathbf{a}) \right]$$

where $\mathbf{h} = \langle h_1, \dots, h_n \rangle$, $\mathbf{a} = \langle a_1, \dots, a_n \rangle \in \mathbb{A} = \times_i \mathbb{A}_i$, and $\mathbf{h}' = \mathbf{hao} = \langle h'_1, \dots, h'_n \rangle$ with $\mathbf{o} = \langle o_1, \dots, o_n \rangle \in \mathbb{O} = \times_i \mathbb{O}_i$. A centralized solution to an MPOMDP would be $\pi_{MPOMDP} : \mathbb{H} \rightarrow \mathbb{A}$ while a decentralized solution would be $\pi_{i, DecentralizedMPOMDP} : \mathbb{H} \rightarrow \mathbb{A}_i$ for each agent i . Like the MMDP case, the difference is the output (i.e., the joint or individual agent actions) and the input (i.e., the joint histories) remains the same. Other POMDP algorithms can be applied to the MPOMDP case by replacing the histories with joint histories, using the joint observations to update the joint histories (i.e., all agents share their information), and replacing the actions with joint actions. Again, additional coordination is needed when solving MPOMDPs in a decentralized manner.

In the other chapters of the book, I will assume the policies are all for Dec-POMDPs so I will drop the subscript indicating what kind of policy it is.

2.4 Improving scalability

While the approaches above appear to be simple, they have one major issue—scalability. As the state and action spaces in the MMDP or the observation and action spaces in the MPOMDP grow, solving these problems becomes computationally intractable. Numerous methods leverage the multi-agent structure to address scalability challenges. I will discuss only a few methods.

One common solution is to assume additional structure in terms of how the agents can coordinate such as through *coordination graphs* [Guestrin et al., 2001]. Coordination graphs decompose the Q-value to be the sum of factors that usually depend on a smaller subset of agents. This additional structure allows maximization of the Q-function without enumerating all actions at a state or history (e.g., using variable elimination [Guestrin et al., 2002]). Some other work explores learning more sparse approximations of the factored Q-function [Kok and Vlassis, 2006a] or extends the idea to Monte Carlo tree search and Bayesian RL [Amato and Oliehoek, 2015]. There have been deep extensions of coordination graphs such as assuming payoffs are between pairs of agents [Böhmer et al., 2020] as well as learning and using more general coordination graphs [Li et al., 2021]. Sometimes these message passing approaches are called *distributed* since communication between neighbors is allowed during execution.

Some high-profile methods have also used centralized training and execution such as AlphaStar [Vinyals et al., 2019] which controls all the units with one centralized agent (unlike the Starcraft Multi-Agent Challenge [Samvelyan et al., 2019]). The large observation space is dealt with using attention and other methods and the large action space is sampled auto-regressively (sequentially) to exploit the multi-agent structure.

For empirical analysis, Gupta et al. [2017] perform comparisons between various centralized, decentralized (i.e., concurrent), and CTDE through parameter sharing.

Chapter 3

Decentralized training and execution (DTE)

3.1 DTE overview

Decentralized training and execution (DTE) is the MARL setting that makes the fewest assumptions. In particular, in the DTE case, at each step of the problem we assume that:

- each agent observes (or maintains) its current history, h_i , takes an action at that history, a_i , and sees the resulting observation, o_i ,
- the (decentralized) algorithm observes the same information (h_i , a_i , and o_i) as well as the joint reward r .

Unlike the CTE case, there is no centralized controller, so the agent must choose actions on its own. In this case, agents only ever observe their own information (actions and observations) and don't observe other agent actions or observations. Similarly, the algorithm can observe the joint reward but (unlike the CTDE case) can only observe the information for that one agent. In DTE, there is strong overlap with the algorithm and agent since the algorithm may specify some aspects of the agent such as exploration but it is still useful to consider them separately to highlight the agent's lack of access to the reward and the optimization role of the algorithm (and to be consistent with other training paradigms). It is worth noting that there are some hidden assumptions (such as concurrent learning) that many DTE algorithms make but I will discuss these in more detail below.

DTE is used in scenarios where centralized information is unavailable or scalability is critical. In particular, due to these limited assumptions, DTE must be used when decentralized agents need to learn without any centralized training phase. That is, if Dec-POMDP agents must learn directly in an environment, agents (and agent optimization algorithms) will not be able to observe centralized information such as other agent actions and observations. As a result, agents will have to learn in a decentralized manner using only the information above. DTE can also be used in settings where the centralization of control, such as in Section 2, or algorithms, such as in Section 4, is computationally intractable. This can occur when the joint history (i.e., observation) or joint action spaces become intractably large.

Because the agent and algorithm information are so well aligned, single-agent RL can also be used here. While the CTE case involved transforming the multi-agent problem into a single agent one by combining all the agents' information, the DTE case naturally fits with the single-agent setting by ignoring the fact that other agents exist and learning as if they were the only agent in the

problem. I will discuss how such methods can be applied in the value-based and policy gradient case as well as improvements to deal with the multi-agent nature of the problem.

3.2 Decentralized, value-based methods

As discussed in Section 1.3.1, value-based methods (such as Q-learning [Watkins and Dayan, 1992]) learn a value function and the policy is implicitly defined based on those values (i.e., choosing the action that maximizes the value function). Some of the earliest MARL methods are based on applying Q-learning to each agent. For instance, Claus and Boutilier [1998] decompose MARL approaches into *independent learners* (IL) and *joint-action learners* (JAL). ILs learn Q-functions that depend on their own information (ignoring other agents) while JALs learn joint Q-functions, which include the actions of the other agents. These concepts were originally defined for the fully observable, bandit (i.e., stateless) case but can be extended to the partially observable, stateful case. For example, we can define an IL Q-function as $Q_i(h_i, a_i)$, which provides a value for agent i 's history h_i and action a_i . A JAL Q-function $Q_i(\mathbf{h}, \mathbf{a})$ for agent i would depend on the joint information $\mathbf{h}$ and $\mathbf{a}$. All decentralized value-based MARL methods are ILs since they never have access to the joint information, $\mathbf{h}$ and $\mathbf{a}$ and many of the algorithms use the ‘independent’ term in their name.

I first discuss the tabular (Q-learning based) methods, starting with a simple application of single-agent Q-learning and then discuss improvements to learning for the multi-agent setting. Next, I discuss extensions to the deep case (DQN-based), again in terms of a simple application of DRQN as well as issues and fixes for the multi-agent case.

3.2.1 Independent Q-learning (IQL)

IQL just applies Q-learning to each agent independently, treating other agents as part of the environment. That is, each agent learns its own local Q-function, Q_i , using Q-learning on its own data. Pseudocode for a version of IQL for the Dec-POMDP case is provided in Algorithm 3. Just like Q-learning in the single-agent case, that algorithm uses a learning rate, α , and some form of exploration, which in this case is ϵ -greedy. Each agent initializes their Q-function (e.g., to 0, optimistically, etc.), and then iterates over episodes (either a fixed set or until convergence). During each episode, the agent chooses an action with exploration, the algorithm sees the joint reward and the agent's own observation and then the algorithm updates the Q-value at the current history, h_i , given the current Q-estimates and the reward using the standard Q-learning update (line 9). The history is updated by appending the action taken and observation seen to the current history and the episode continues until the horizon is reached.

The Q-functions are inherently tied to the history length, which defines the current time step, making the algorithm suitable for finite-horizon cases. Regardless, the algorithm can easily be extended to the episodic/indefinite-horizon case by including terminal states (or histories) instead of a fixed horizon.

Important hidden information This algorithm runs in the underlying Dec-POMDP but the details of those dynamics in the environment are not shown (as they are not visible to the agent and algorithm). For example, at the beginning of each episode, the state, s , is sampled from the initial state distribution b_0 , and when agents take joint action $\mathbf{a}$ the reward is generated from the

Algorithm 3 Independent Q-Learning for agent i (finite-horizon)

```

1: set  $\alpha$  and  $\epsilon$  (learning rate, exploration)
2: Initialize  $Q_i$  for all  $h_i \in \mathbb{H}_i, a_i \in \mathbb{A}_i$ 
3: for all episodes do
4:    $h_i \leftarrow \emptyset$  {Empty initial history}
5:   for  $t = 1$  to  $\mathcal{H}$  do
6:     Choose  $a_i$  at  $h_i$  from  $Q_i(h_i, \cdot)$  with exploration (e.g.,  $\epsilon$ -greedy)
7:     See joint reward  $r$ , local observation  $o_i$  {Depends on joint action  $\mathbf{a}$ }
8:      $h'_i \leftarrow h_i a_i o_i$ 
9:      $Q_i(h_i, a_i) \leftarrow Q_i(h_i, a_i) + \alpha [r + \gamma \max_{a'_i} Q_i(h'_i, a'_i) - Q_i(h_i, a_i)]$ 
10:     $h_i \leftarrow h'_i$ 
11:   end for
12: end for
13: return  $Q_i$ 

```

reward function, $R(s, \mathbf{a})$, the next state s' is sampled from the transition dynamics with probability $\Pr(s'|s, \mathbf{a})$, and the joint observation is sampled from the observation function $\Pr(\mathbf{o}|s', \mathbf{a})$. This process continues until the end of the episode.

While independent Q-learning ignores the other agents, it still depends on them in order to generate the joint reward r and the local observation o_i since both of these depend on the actions of all agents. That is, from agent i 's perspective, it is learning in a (single-agent) POMDP, as Algorithm 3 shows (the states of the POMDP would be the states of the Dec-POMDP plus the histories of the other agents).¹

In particular, as pointed out in previous papers [Lauer and Riedmiller, 2000]², independent Q-learning is actually learning a Q-function for each agent based on the actions selected by the other agents during training. That is, agent i would use data that depends on the actions taken by the other agents (and observations other agents see) even if it doesn't directly observe those actions (and observations). In the partially observable case, the following Q-function would be learned:

$$Q_i(h_i, a_i) = \sum_{\mathbf{a} \in \mathbb{A}} \hat{P}(\mathbf{a}, \mathbf{h} | h_i, a_i) \left[r + \gamma \sum_{o_i} \hat{P}(o_i | \mathbf{h}, \mathbf{a}) \max_{a'_i} Q_i(h'_i, a'_i) \right] \quad (3.1)$$

where $P(\mathbf{a}, \mathbf{h} | h_i, a_i)$ is the empirical probability that the joint action $\mathbf{a}$ and joint history $\mathbf{h}$ occurs when agent i selects its action a_i in h_i . This is precisely when independent Q-learning will update $Q_i(h_i, a_i)$ and it will use the joint reward r and agent's observation function $P(o'_i | \mathbf{a}, \mathbf{h})$, which approximates $P(o'_i | \mathbf{a}, s')P(s' | \mathbf{a}, s)$, where $P(o'_i | \mathbf{a}, s')$ marginalizes out other agent observations from the joint function $P(\mathbf{o}' | \mathbf{a}, s')$. $\hat{P}(o_i | \mathbf{h}, \mathbf{a})$ is the empirical probability based on observing o_i from $\mathbf{h}$ after taking action $\mathbf{a}$. Therefore, IQL is assuming the other agents are part of the environment and learning a Q-function for the resulting POMDP. If the other agents are not learning and do in

¹The idea of fixing other agent policies and then solving the resulting POMDP (either by planning methods [Nair et al., 2003] or RL methods [Banerjee et al., 2012]) has had success and results in a best response for agent i . Furthermore, iterating over agents, i , fixing other agent policies, $\neg i$, and calculating best responses will lead to a local optimum of the Dec-POMDP [Banerjee et al., 2012]. Unfortunately, such methods require coordination to communicate which agent's turn it is to learn so they are not appropriate for the DTE case.

²Note the paper considers the fully observable deterministic transition case but I extend the idea to the Dec-POMDP case.

fact have fixed policies, the problem would just reduce to this POMDP. Unfortunately, the other agents would also typically be learning, leading to *nonstationarity* in the underlying POMDP and difficulty with coordinated action selection.

Convergence and solutions In the case of IQL, the underlying POMDP that is being learned is nonstationary since the other agents are also learning and thus changing their policies over time. This may cause IQL to not converge [Tan, 1993]. Convergence of Q-learning in the multi-agent case is an active area of research [Claus and Boutilier, 1998, Tuyls et al., 2003, Wunder et al., 2010, Hussain et al., 2023], it is still an open question what assumptions will allow convergence and to what (unlike the policy gradient case where convergence to a local optimum is assured under mild assumptions [Peshkin et al., 2000, Lyu et al., 2023])³.

It is worth pointing out that even if algorithms can converge to an optimal Q-value (or any Q-value), agents may not be able to select optimal actions if there are multiple optimal policies [Lauer and Riedmiller, 2000]. In this case, agents would need to coordinate on their actions to make sure they choose actions from the same joint policy but this is not possible with only individual Q-values (e.g., if $Q_1(h_1, a_1^1) = Q_1(h_1, a_1^2)$ and $Q_2(h_2, a_2^1) = Q_2(h_2, a_2^2)$ but $Q(h_1, h_2, a_1^1, a_2^2) = Q(h_1, h_2, a_1^2, a_2^1) < Q(h_1, h_2, a_1^2, a_2^2) = Q(h_1, h_2, a_1^1, a_2^1)$ —agents must take the same actions to be optimal).

IQL is very simple but can perform well [Tan, 1993, Sen et al., 1994, Tampuu et al., 2017]. It is also the basis for the other value-based DTE MARL methods.

3.2.2 Improving the performance of IQL

Several extensions of IQL have been developed to improve performance and better fit with the MARL setting. While the below algorithms were developed for the fully observable case (the multi-agent MDP described in Section 2, I extend them to the partially observable case here to fit better with our Dec-POMDP setting.

Distributed Q-learning

In order to promote coordination, Distributed Q-learning [Lauer and Riedmiller, 2000] makes strong optimistic assumptions, prioritizing high-reward outcomes from agent actions while discounting suboptimal ones during learning. In particular, extending to the partially observable case, it uses the following Q-update:

$$Q_i(h_i, a_i) = \max \left\{ Q_i(h_i, a_i), r + \gamma \max_{a'_i} Q_i(h'_i, a'_i) \right\} \quad (3.2)$$

where the current Q-value is kept if it is higher than (or equal to) the new estimate $r + \gamma \max_{a'_i} Q_i(h'_i, a'_i)$. Otherwise, the Q-value is updated to be the new estimate. The intuition behind this optimistic update is that other agents will make mistakes during training because they have suboptimal policies or because they are exploring. Agents don't necessarily want to learn from these mistakes. Instead, agents want to learn from the best policies of the other agents such as those that choose actions

³In fact, while Q-learning can provably converge to a global optimum in the single-agent case [Jaakkola et al., 1993], there are no known MARL algorithms that have guaranteed convergence to a global optimum. This is an interesting area for future research!

that generate high-reward values, even if they don't happen very often. In the special case of deterministic MMDPs, this approach will learn the same Q-function as centralized Q-learning (which will converge to the optimal Q-values in the limit). This is unlike IQL which can get stuck in local optima or never converge at all.

Because Distributed Q-learning learns individual Q-values, agents may still not be able to select optimal actions due to the coordination issue mentioned above. As a result, Distributed Q-learning stores the current best policy for each agent (i.e., if the Q-value is updated, the action that resulted in that update is stored for that state in the fully observable case or history in the partially observable case). Including this coordination mechanism, the algorithm can converge to optimal Q-functions and optimal policies in the limit for the fully observable deterministic case.

In the more general stochastic case, distributed Q-learning is not able to distinguish between environmental stochasticity and agent stochasticity. That is, if getting a high-valued reward is unlikely (due to reward or transition stochasticity) distributed Q-learning will assume it can be received deterministically using max rather than taking the expectation (over other agent policies and stochastic transitions) as in Equation 3.1. As a result, distributed Q-learning is overly optimistic in the stochastic case and may perform poorly. Learning and incorporating the probabilities in Equation 3.1 and then calculating the expectation over the next step Q-value could fix this problem but would assume agents observe the actions of other agents which is not the case for decentralized training.

Hysteretic Q-learning

To maintain the idea of optimism while accounting for stochasticity, hysteretic Q-learning [Matignon et al., 2007] was developed. The idea is to use two learning rates, α and β with $\alpha > \beta$. That is, agents update their Q-values more during a positive experience than a negative one.

In particular, if we define the TD error as:

$$\delta \leftarrow r + \gamma \max_{a'_i} Q(h'_i, a'_i) - Q_i(h_i, a_i),$$

then we can rewrite distributed Q-learning (using a learning rate α) as:

$$Q_i(h_i, a_i) = \begin{cases} Q_i(h_i, a_i) + \alpha\delta & \text{if } \delta > 0 \\ Q_i(h_i, a_i) & \text{else} \end{cases} \quad (3.3)$$

highlighting the fact that positive experiences (in terms of TD error) cause updates to the Q-function while negative ones do not.

Hysteretic Q-learning, in contrast, adds the second learning rate β that is applied to the negative experiences:

$$Q_i(h_i, a_i) = \begin{cases} Q_i(h_i, a_i) + \alpha\delta & \text{if } \delta > 0 \\ Q_i(h_i, a_i) + \beta\delta & \text{else} \end{cases} \quad (3.4)$$

This ensures learning still occurs in the negative cases but the update is less than in the positive cases because $\alpha > \beta$. Therefore, hysteretic Q-learning is still optimistic (hoping the positive experiences are because of better action choices and not stochasticity) but is less optimistic than distributed Q-learning. The implementation is also very simple (although it does require tuning two learning rates). Hysteretic Q-learning is very simple but can perform well in a range of domains [Matignon et al., 2007]. Also, note that as β approaches α , hysteretic Q-learning will become standard Q-learning and as β approaches 0, it will become distributed Q-learning.

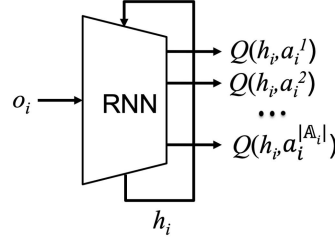


Figure 3.1: Independent DRQN diagram.

Lenient Q-learning

While it is possible to adjust β towards α as learning continues, to be more optimistic at the beginning of learning (when a lot of exploration is happening) and still be robust to stochasticity as learning converges, lenient Q-learning [Wei and Luke, 2016] allows agents to adjust this *leniency* towards exploration on a history-action pair basis. Specifically, lenient Q-learning maintains a temperature, $T(h_i, a_i)$, per history-action pair that adjusts the probability that a negative update at a particular history-action pair will be ignored. These temperature values allow history-action pairs that have not been visited often to not be updated yet, while frequently visited history-action pairs are updated as in IQL.

The lenient Q-learning update is:

$$Q_i(h_i, a_i) = \begin{cases} Q_i(h_i, a_i) + \alpha \delta & \text{if } \delta > 0 \text{ or } rand \sim U(0, 1) > 1 - e^{-K * T(h_i, a_i)} \\ Q_i(h_i, a_i) & \text{else} \end{cases} \quad (3.5)$$

$T(h_i, a_i)$ is initialized to a maximum temperature, and is decayed after an update with $T(h_i, a_i) \leftarrow \lambda T(h_i, a_i)$ for $\lambda \in (0, 1)$ and leniency parameter K adjusts the impact of the temperature on the probability distribution. The resulting Q-update will probabilistically update Q based on how often history-action pairs are visited, with more frequently visited pairs being more likely to be updated when the TD error is negative.

Lenient Q-learning can outperform hysteretic Q-learning by fine-tuning optimism levels on a per-state-action basis. Unfortunately, it also requires maintaining (and updating) temperature values as well as Q-values. Maintaining these values can be more problematic in the partially observable case since the history-action space grows exponentially with the problem horizon (or history length more generally).

3.2.3 Deep extensions, issues, and fixes

In order to scale to larger domains, deep extensions of the above algorithms have been developed. These approaches are typically based on deep Q-networks (DQN) [Mnih et al., 2015]. While DQN can scale to larger action and observation spaces, the decentralized multi-agent context can cause challenges such as non-stationarity and coordination difficulties. I discuss a deep version of IQL, independent DRQN [Tampuu et al., 2017], the coordination issues with the approach, and some proposed fixes. Details about DQN and DRQN are in Section 1.3.1.

Independent DRQN (IDRQN)

Independent DRQN (IDRQN) [Tampuu et al., 2017]⁴ combines IQL and DRQN. As seen in Algorithm 4, the basic structure is the same as IQL but Q_i is estimated using an RNN parameterized by θ (shown in Figure 3.1), along with using a target network, θ^- , and a replay buffer, $\mathcal{D}$. First, episodes are generated by interacting with the environment (e.g., ϵ -greedy exploration), which are added to the replay buffer. This buffer typically has a fixed size and new data replaces old data when the buffer is full. Episodes can then be sampled from the replay buffer (either single episodes or as a minibatch of episodes). In order to have the correct internal state, RNNs are trained sequentially. Training is done from the beginning of the episode until the end⁵, calculating the internal state and the loss at each step and updating θ using gradient descent. The target network θ^- is updated periodically (every C episodes in the code) by copying the parameters from θ .

Again, the code can be adjusted to the episodic/indefinite-horizon case by replacing the horizon with terminal states or the infinite-horizon case by removing the loop over episodes.

Issues with IDRQN Independent DRQN can perform well but has a number of issues. For example, early methods disabled experience replay [Foerster et al., 2016]⁶. They found that performance was worse with experience replay than without it. This is because agents are no longer learning concurrently when using the replay buffer—experience is generated from the behavior of all agents (i.e., concurrently) but is then put into the agent’s buffer for learning. When learning, different agents will sample from their buffers separately, learning from different experiences. Learning from this different data makes performance noisy and unstable. This makes intuitive sense since agents will update their Q-functions (and resulting policies) based on (potentially very) different experiences. I discuss a simple fix next.

Deep hysteretic-Q and concurrent buffer

Hysteretic Q-learning has been extended to the deep case in the form of decentralized hysteretic deep recurrent Q-networks (Dec-HDRQN) [Omidshafiei et al., 2017]. Dec-HDRQN makes two main contributions: the use of hysteresis with D(R)QN, and a concurrent replay buffer to improve performance.⁷

The inclusion of hysteresis in Dec-HDRQN just combines hysteresis and IDRQN. That is, since $y - Q_i^\theta(h_i, a_i)$ in the IDRQN loss is the TD error, δ , the network parameters are updated (using gradient descent) using learning rate α if $\delta > 0$, and β is used otherwise.

The concurrent buffer is also relatively straightforward. Concurrent experience replay trajectories (CERTs), are based on the idea of creating a joint experience replay buffer and then sampling from that buffer. In particular, we would like to have a joint replay buffer with joint experience tuples, $\mathbf{a}_1^e, \mathbf{o}_1^e, r_1^e, \dots, \mathbf{a}_t^e, \mathbf{o}_t^e, r_t^e$, where the superscript here represents the episode number, e . Agents could then sample from this joint buffer to get the local information from the same episode and time step: h_i, a_i, o_i, r (where h_i represents the history at the current time step such as the internal

⁴Again, the original algorithm only considers the fully observable case but I consider the extension to the Dec-POMDP case.

⁵It is common in practice to use a fixed history length and sample windows of that length rather than training with full episodes.

⁶Other methods for improving experience replay in MARL exist, but they typically assume the CTDE setting (such as Foerster et al. [2017]).

⁷The paper also discusses the multi-task case, but I don’t discuss that aspect here.

Algorithm 4 Independent DRQN (IDRQN) for agent i (finite-horizon*)

```

1: set  $\alpha$ ,  $\epsilon$ , and  $C$  (learning rate, exploration, and target update frequency)
2: Initialize network parameters  $\theta$  and  $\theta^-$  for  $Q_i$ 
3:  $\mathcal{D}_i \leftarrow \emptyset$ 
4:  $e \leftarrow 1$  {episode index}
5: for all episodes do
6:    $h_i \leftarrow \emptyset$  {initial history is empty}
7:   for  $t = 1$  to  $\mathcal{H}$  do
8:     Choose  $a_i$  at  $h_i$  from  $Q_i^\theta(h_i, \cdot)$  with exploration (e.g.,  $\epsilon$ -greedy)
9:     See joint reward  $r$ , local observation  $o_i$  {Depends on joint action  $\mathbf{a}$ }
10:    append  $a_i, o_i, r$  to  $\mathcal{D}_i^e$ 
11:     $h_i \leftarrow h_i a_i o_i$  {update RNN state of the network}
12:   end for
13:   sample an episode from  $\mathcal{D}$ 
14:   for  $t = 1$  to  $\mathcal{H}$  do
15:      $h_i \leftarrow \emptyset$ 
16:      $a_i, o_i, r \leftarrow \mathcal{D}_i^e(t)$ 
17:      $h'_i \leftarrow h_i a_i o_i$ 
18:      $y = r + \gamma \max_{a'_i} Q_i^{\theta^-}(h'_i, a'_i)$ 
19:     Perform gradient descent on parameters  $\theta$  with learning rate  $\alpha$  and loss:  $(y - Q_i^\theta(h_i, a_i))^2$ 
20:      $h_i \leftarrow h'_i$ 
21:   end for
22:   if  $e \bmod C = 0$  then
23:      $\theta^- \leftarrow \theta$ 
24:   end if
25:    $e \leftarrow e + 1$ 
26: end for
27: return  $Q_i$ 

```

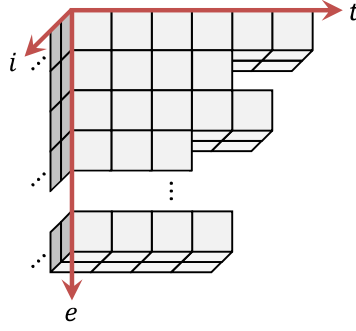


Figure 3.2: Concurrent experience replay trajectories (CERTs) (from [Omidshafiei et al., 2017])

state of agent i 's RNN after passing the local history of actions and observations until the time step being sampled). Of course, in a decentralized learning context, agents will not have access to such a joint buffer, but agents can store their part of the joint buffer by indexing by episode e and timestep t . That is, as seen in Figure 3.2, agents store their local trajectories along axes for e and t . When an e and t is sampled, each agent i will sample data from the same episode and timestep, producing the same data as would be produced with the joint buffer. As long as agents have the same size buffers and coordinate on the seeds for the random number generator, this approach can be used for decentralized learning.

Deep lenient Q-learning

Lenient Q-learning has been extended to the *fully-observable* deep case in the form of lenient DQN [Palmer et al., 2018]. In this case, the leniency values are added to the replay buffer of DQN at each step as: $(s_t, a_t, r_t, s_{t+1}, l(s_t, a_t))$ where $l(s_t, a_t) = 1 - e^{-K \cdot T(\phi(s_t), a_t)}$. To scale to large state spaces, the approach clusters states using an autoencoder which outputs a hash $\phi(s)$ for a given state s . The method also introduces a temperature decay schedule after reaching the terminal state and an improved exploration scheme based on average temperature value.

The approach was shown to perform well in a set of coordinated multi-agent object transportation problems (CMOTPs). It was not extended to the partially observable case but it should be possible by using a method that clusters histories rather than states.

Likelihood Q-learning

Distributional Q-learning [Bellemare et al., 2023] has also been used to improve decentralized MARL. Instead of just learning the expected Q-value as in traditional Q-learning, distributional RL learns the distribution of returns that are possible due to stochasticity. Likelihood Q-learning [Lyu and Amato, 2020] uses the return distribution to determine the likelihood of the experience (i.e., h_i, a_i, o_i, r) given the current estimate, which they call Time Difference Likelihood (TDL). This information can be used to help determine if other agents are exploring (i.e., low likelihood and low value). As a result, the TDL can be used as a multiplier of the learning rate, causing larger updates for more likely experiences—when other agents are not exploring.

But some low-likelihood experiences may be good to learn from, such as when agents have found a high-quality joint action (i.e., low likelihood but high value). Distributional information can then be used to estimate risk [Morimura et al., 2010]. In the multi-agent context risk-seeking behavior can mean being optimistic as in hysteretic or lenient learning.

Each of these ideas can be used in isolation or in combination and they can be used with other decentralized MARL methods to further improve performance. For instance, the combination of Dec-HDRQN (see 3.2.3) with TDL was shown to outperform Dec-HDRQN and forms of lenient Q-learning. Incorporating risk can further improve performance.

3.3 Decentralized policy gradient methods

Single-agent policy gradient and actor-critic methods can similarly be extended to the decentralized training multi-agent case by allowing each agent to learn separately. These approaches can scale to larger action spaces and have stronger (local) convergence guarantees than the value-based counterparts above. Background on single-agent policy gradient methods is in Section 1.3.2.

Policy gradient methods can be used with continuous-action or stochastic policies. For all approaches, I will assume that we have a stochastic policy for each agent parameterized by ψ_i , where $\pi_i^{\psi_i}(a_i|h_i)$ represents the probability agent i will choose action a_i given the history h_i and parameters, ψ_i , $\Pr(a_i|h_i, \psi_i)$. This is in contrast to the value-based methods in Section 3.2, where deterministic policies were generated based on the learned value functions.

3.3.1 Decentralized REINFORCE

As mentioned in Section 1.3.2, REINFORCE [Williams, 1992] is one of the oldest (and simplest) policy gradient methods. The basic idea is to estimate the value of the policy using Monte Carlo rollouts and then update the policy using gradient ascent. REINFORCE was extended to the cooperative multi-agent case by Peshkin et al. [2000]. Importantly, Peshkin et al. [2000] also showed that the joint gradient can be decomposed into decentralized gradients. That is, decentralized gradient ascent will be equivalent to joint gradient ascent. Convergence to a local optimum is guaranteed when agents are learning concurrently (each agent is learning on the data that is generated by all agents and all agents make updates at the same time) because they will apply the same algorithm to the same data at synchronized time steps. This convergence result is in contrast to the value-based methods, which do not have strong convergence guarantees. These convergence guarantees extend to many other policy gradient methods (as discussed below).

Pseudocode for a version of decentralized REINFORCE is in Algorithm 5. Here, a set of stochastic policies, π_i , that are parameterized by ψ_i (e.g., parameters of a neural network), are learned. A learning rate of α is used (as usual). The method iterates for some number of episodes, choosing actions from the stochastic policy (which has positive probability for all actions to ensure proper exploration), observing the joint reward and local observation at each time step, and storing the episode. Then, the return is computed for each step of the episode and the parameters of the policy are updated using the returns and gradient (just like the single-agent version of REINFORCE). Time steps are explicitly represented in the code to make return calculation clear. Again, the method can be extended to the episodic/indefinite-horizon case by replacing the horizon with terminal states and it isn't well-defined for the infinite-horizon case (since updating takes place at the end of episodes).

Decentralized REINFORCE isn't widely used but serves as a basis for the later policy gradient approaches as well as theoretical guarantees.

Algorithm 5 Decentralized REINFORCE for agent i (finite-horizon)**Require:** Individual actor models $\pi_i(a_i|h_i)$, parameterized by ψ_i

```

1: set  $\alpha$  (learning rate)
2: for all episodes do
3:    $h_{i,0} \leftarrow \emptyset$  {Empty initial history}
4:    $ep \leftarrow \emptyset$  {Empty episode}
5:   for  $t = 0$  to  $\mathcal{H} - 1$  do
6:     Choose  $a_{i,t}$  at  $h_{i,t}$  from  $\pi_i(a_i|h_{i,t})$ 
7:     See joint reward  $r_t$ , local observation  $o_{i,t}$  {Depends on joint action  $\mathbf{a}$ }
8:     append  $a_{i,t}, o_{i,t}, r_t$  to  $ep$ 
9:      $h_{i,t+1} \leftarrow h_{i,t}a_{i,t}o_{i,t}$  {Append new action and obs to previous history}
10:  end for
11:  for  $t = 0$  to  $\mathcal{H} - 1$  do
12:    Compute return at  $t$  from  $ep$ :  $G_{i,t} \leftarrow \sum_{k=t}^{\mathcal{H}-1} \gamma^{k-t} r_k$ 
13:    Update parameters:  $\psi_i \leftarrow \psi_i + \alpha \gamma^t G_{i,t} \nabla \log \pi_i(a_i|h_{i,t})$ 
14:  end for
15: end for

```

3.3.2 Independent actor critic (IAC)

Since REINFORCE is a Monte Carlo method, it must wait until the end of an episode to update. Furthermore, it will typically be less sample efficient than methods that learn a value function. As mentioned in Section 1.3.2, actor-critic methods have been developed to combat these issues by learning a policy representation (the actor) as well as a value function (the critic) [Sutton and Barto, 2018]. Actor-critic methods can use the critic to evaluate the actor without waiting until the end of the episode and TD-learning to update the critic more efficiently.

Independent actor-critic (IAC) was first introduced by Foerster et al. [2018b] and it can be thought of as a combination of IQL (or IDRQN) and decentralized REINFORCE. Each agent learns a policy, $\pi_i(a_i|h_i)$, as well as a value function such as $Q_i(h_i, a_i)$. In the simplest case of IAC (where a Q-value is used for the critic), the gradient associated with the policy parameters ψ_i can be derived as

$$\nabla_{\psi_i} J = \mathbb{E}_{\mathbf{h}, \mathbf{a} \sim \mathcal{D}} \left[Q_i(h_i, a_i) \nabla_{\psi_i} \log \pi_i^{\psi_i}(a_i|h_i) \right],^8 \quad (3.6)$$

assuming a stochastic policy for each agent i that is parameterized by ψ_i , $\pi_i^{\psi_i}(a_i|h_i)$. Policies are sampled according to the on-policy (discounted) visitation probability (which is a joint visitation probability since concurrent sampling is assumed). This gradient is very similar to the REINFORCE one but uses a Q-value rather than the Monte Carlo return (and is shown here with the expectation rather than just a sample).

In the on-policy, function-approximation (e.g., deep) case, the critic can be updated as:

$$\mathcal{L}(\theta_i) = \mathbb{E}_{\langle h_i, a_i, r, o_i \rangle \sim \mathcal{D}} \left[(y - Q_i^{\theta_i}(h_i, a_i))^2 \right], \text{ where } y = r + \gamma Q_i^{\theta_i}(h'_i, a'_i) \quad (3.7)$$

where $h'_i = h_i a_i o_i$ and a'_i is the action sampled at the next time step using $\pi_i^{\psi_i}(a'_i|h'_i)$ since it is an

⁸For simplicity and to match common implementations, we ignore discounting but the true gradient would include it [Nota and Thomas, 2020].

on-policy estimate (rather than the max action used in DQN). $Q_i^{\theta_i}(h_i, a_i)$ represents the Q-function estimate for agent i using parameters θ_i (just like in Section 3.2).

Pseudocode for IAC is given in Algorithm 6. Because the value function is no longer being used to select actions but rather just to evaluate the policy, it is common to instead learn $V_i(h_i)$. And because a model of V is learned, I denote it $\hat{V}$. Like the REINFORCE case, the algorithm loops for some number of episodes, choosing actions from the stochastic policy and seeing the joint reward and local observation at each step. Now, during the episode, the TD error is calculated and used to update the actor and the critic using the associated gradients described above. Since V-values are learned instead of Q-values, the TD error is calculated with $r_t + \gamma \hat{V}_i(h_{i,t+1}) - \hat{V}_i(h_{i,t})$ rather than Q_i . As a result, a value-based version of Equation 3.6 is used to update the actors. A value-based version of Equation 3.7 is recovered by using $y = r_t + \gamma \hat{V}_i(h_{i,t+1})$ and replacing $Q_i^{\theta_i}(h_i, a_i)$ with $\hat{V}_i(h_{i,t})$.

Algorithm 6 Independent Actor-Critic (IAC) (finite-horizon)

Require: Individual actor models $\pi_i(a_i|h_i)$, parameterized by ψ_i

Require: Individual critic models $\hat{V}_i(h)$, parameterized by θ_i

```

1: for all episodes do
2:    $h_{i,0} \leftarrow \emptyset$  {Empty initial history}
3:   for  $t = 0$  to  $\mathcal{H} - 1$  do
4:     Choose  $a_{i,t}$  at  $h_{i,t}$  from  $\pi_i(a_i|h_{i,t})$ 
5:     See joint reward  $r_t$ , local observation  $o_{i,t}$  {Depends on joint action a}
6:      $h_{i,t+1} \leftarrow h_{i,t} a_{i,t} o_{i,t}$  {Append new action and obs to previous history}
7:     Compute value TD error:  $\delta_{i,t} \leftarrow r_t + \gamma \hat{V}_i(h_{i,t+1}) - \hat{V}_i(h_{i,t})$ 
8:     Compute actor gradient estimate:  $\gamma^t \delta_{i,t} \nabla \log \pi_i(a_{i,t}|h_{i,t})$ 
9:     Update actor parameters  $\psi_i$  using gradient estimate (e.g.,  $\psi_i \leftarrow \psi_i + \alpha \gamma^t \delta_{i,t} \nabla \log \pi_i(a_{i,t}|h_{i,t})$ )
10:    Compute critic gradient estimate:  $\delta_{i,t} \nabla \hat{V}_i(h_{i,t})$ 
11:    Update critic parameters  $\theta_i$  using gradient estimate (e.g.,  $\theta_i \leftarrow \theta_i + \beta \gamma \delta_{i,t} \nabla \hat{V}_i(h_{i,t})$ )
12:   end for
13: end for

```

Like the other algorithms, IAC can be adapted to the episodic or infinite-horizon case by including terminal states or removing the loop over episodes. IAC is a simple baseline that is often used and serves as the basis of many other decentralized MARL actor-critic methods.

3.3.3 Other decentralized policy gradient methods

Any single-agent method can be extended to the decentralized MARL case. Typically, it just entails including recurrent layers to deal with partial observability and then training agents concurrently to generate joint data and synchronize updates. One notable example is IPPO [de Witt et al., 2020, Yu et al., 2022]. IPPO is the extension of single-agent PPO [Schulman et al., 2017] to the multi-agent case. Forms of IPPO have been shown to outperform many state-of-the-art MARL methods. It is worth noting that the standard implementations of IPPO (and the impressive results) use parameter sharing (as discussed below) so they are not technically decentralized but decentralized implementations can still perform well [Yu et al., 2022]. I discuss IPPO (and MAPPO) in more detail in Section 4.3.5 and discuss parameter sharing below.

3.4 Other topics

Concurrent learning It is very important to note that all of the methods in this section assume agents are running the same algorithm and perform synchronous updates (i.e., at the same time on the same data). Without these assumptions, methods that have convergence guarantees lose them but may still work well in some domains. As discussed in relation to DQN (see Section 3.2.3), breaking this ‘concurrent’ learning assumption can cause methods to perform poorly.

Parameter sharing Parameter (or weight) sharing, where agents share the same network for estimating the value function in value-based methods or policies (and value functions) in policy gradient methods, is common in cooperative MARL. The data from each agent can be used to update a single value network for an algorithm such as DRQN. Agents can still perform differently due to observing different histories and agent indices can be added to increase specialization [Gupta et al., 2017, Foerster et al., 2016]. While parameter sharing is typically used with homogeneous agents (i.e., those with the same action and observation spaces), it can also be used with heterogeneous agents [Terry et al., 2020]. Since parameter sharing requires agents to share networks during training, it isn’t strictly DTE as it couldn’t be used for online training in a decentralized fashion. Nevertheless, any of the algorithms in this chapter can be extended to use parameter sharing. The resulting algorithms would be a form of CTDE as the shared network would represent a centralized component. Nevertheless, these parameter-sharing implementations may be more scalable than other forms of CTDE, are often simple to implement, and can perform well [Gupta et al., 2017, Foerster et al., 2018b, Yu et al., 2022].

Relationship with CTDE While CTDE methods are (by far) the most popular form of MARL, they do not always perform the best. In fact, since DTE methods typically make the concurrent learning assumption, they are actually quite close to CTDE methods. This is shown explicitly in the policy gradient case as the gradient of the joint update is the same as the decentralized gradient (as described above in Section 3.3.1)[Peshkin et al., 2000]. This phenomenon has been also been studied theoretically and empirically with modern actor-critic methods [Lyu et al., 2021, 2023]. It turns out that while centralized critic actor-critic methods are often assumed to be better than DTE actor-critic methods, they are theoretically the same (with mild assumptions) and often empirically similar (and sometimes worse). Too much centralized information can sometimes be overwhelming, harming scalability of centralized-critic methods [Yu et al., 2022, Lyu et al., 2023]. Furthermore, some CTDE methods, such as using a state-based critic, are fundamentally unsound and can often hurt performance in partially observable domains [Lyu et al., 2022, 2023]. More details about these issues can be found in Sections 4.3.7 and 4.3.6. Of course, there are many different ways of performing centralized training for decentralized execution, and studying the differences and similarities between DTE and CTDE variants of algorithms seems like a promising direction for understanding current methods and developing improved approaches for both cases.

Other similar types of learning not considered here There are a number of papers that focus on ‘fully’ decentralized MARL, such as Zhang et al. [2018]. These methods assume communication between agents during training and execution. As a result, the assumptions are different than the ones considered here. I would term these ‘fully’ decentralized as distributed or networked MARL to make this point more clear. There are also methods that assume additional coordination. For

example, some methods assume agents take turns learning while the other agents remain fixed [Su et al., 2024, Banerjee et al., 2012]. Such methods require coordinating updates but can converge to local (i.e., Nash) equilibria. More details about this alternating learning case are discussed in Section 4.4.

Chapter 4

Centralized training for decentralized execution (CTDE)

4.1 CTDE overview

The idea of centralized training for decentralized execution is quite vague. The general idea of CTDE in the Dec-POMDP case originated (with the Dec-POMDP itself) in the planning literature [Bernstein et al., 2002], where *planning* would be centralized but execution was decentralized. This idea goes back further to *team* decision making more generally since it is natural to think of deriving a solution for the team as a whole and then assigning corresponding parts to the team members [Marschak, 1955, Radner, 1962, Ho, 1980]. This concept was carried over to the reinforcement learning case and has come to mean that there is an ‘offline’¹ training phase where some amount of centralization is allowed (such as in [Kraemer and Banerjee, 2016]). Typically, this centralized training phase is assumed to take place using a simulator where centralized information is available while execution takes place in the actual decentralized environment. After this centralized training phase is complete, agents must then act in a decentralized manner based on their own histories (as given by the Dec-POMDP definition above). Any information that would not be available during decentralized execution can be used in the centralized training phase ranging from neural network parameters, policies of the other agents, a joint value estimate, etc. Nevertheless, CTDE has never been formally defined (to the best of my knowledge). I will take a wide view of CTDE and consider it to include any centralized information or coordination during learning—anything that is not available during decentralized execution. Admittedly, there is a bit of a gray area between DTE and CTDE where methods such as deep hysteretic Q-learning that coordinate on a set of random seeds to use during execution could be considered DTE or CTDE.

In the CTDE case, at each step of the problem we assume that:

- each agent observes (or maintains) its current history, h_i , takes an action at that history, a_i , and sees the resulting observation, o_i ,
- the (centralized) algorithm observes the *joint* information from all agents ($\mathbf{h}$, $\mathbf{a}$, and $\mathbf{o}$) as well as the joint reward r (and possibly other information such as the underlying state s).

¹Offline does not refer to having a fixed training set [Levine et al., 2020] but rather in a different setting than what is used for execution (which would be considered online).

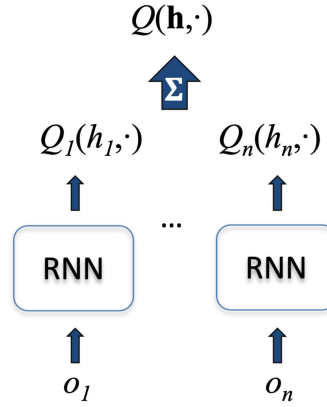


Figure 4.1: VDN architecture diagram

CTDE methods for Dec-POMDPs vary widely. Some methods add centralized information to DTE methods. These approaches should be scalable but use limited centralized information. Other methods are at the other extreme, where they learn a centralized policy and then attempt to decentralize it so it can be executed without the centralized information. Most CTDE methods fall somewhere in between these two ideas. The main class of value-based CTDE methods learns a set of decentralized Q-functions (one for each agent) by factoring a joint value function, as in value function factorization methods. The main class of policy gradient CTDE methods learns a centralized critic that approximates the joint value function for all agents and that centralized critic is used to update a set of decentralized actors (one per agent). Because most of the work has focused on deep reinforcement learning versions of these ideas, I will discuss the methods and implementations from that perspective.

4.2 Value function factorization methods

As mentioned in Section 2.4, there is a long history of learning factored value functions in multi-agent reinforcement learning, but many previous methods were focused on more efficiently learning a joint value function (e.g., [Kok and Vlassis, 2006b]). Other methods have also been developed that share Q-functions during training to improve coordination [Schneider et al., 1999]. Modern value function factorization methods learn Q-functions for each agent by combining them into a joint Q-function and calculating a loss based on the joint Q-function. In this way, the joint Q-function is factored into local, decentralized Q-values that can be used during decentralized execution. This idea is appealing because a joint Q-function can be learned to approximate the one in Equation 1.2 but agents can choose actions based on their decentralized Q-values. Additional details and the most popular methods are below.

4.2.1 VDN

Value decomposition networks (VDN) [Sunehag et al., 2017] began the popular trend in value factorization methods for solving deep multi-agent reinforcement learning. The main contribution

in VDN is a decomposition of the joint Q-function into additive Q-functions per agent.² This idea is relatively simple but very powerful. It allows training to take place in a centralized setting but agents can still execute in a decentralized manner because they learn individual Q-functions per agent, which can then be maxed over to select an action.

In particular, it assumes the following approximate factorization of the Q-function:

$$Q(\mathbf{h}, \mathbf{a}) \approx \sum_{i \in \mathbb{I}}^n Q_i(h_i, a_i) \quad (4.1)$$

That is, the joint Q-function is approximated by a sum over each agent’s individual Q-function. This is a form of factorization—the joint Q-function is factored as a sum of the local Q_i functions [Guestrin et al., 2001]. It is worth noting that each Q_i is technically a utility and not a value function since it is not estimating a particular expected return but can be any arbitrary value in the sum.

The architecture diagram is given in Figure 4.1. Each agent learns an individual set of Q-values that depend only on local information. These Q-networks take the current observation as input to a recurrent network, which updates the history representation and then outputs Q-values for that agent. During training, these individual Q-values are summed to generate the joint Q-value which can then be used to calculate the loss like DRQN:

$$\mathcal{L}(\theta) = \mathbb{E}_{\langle \mathbf{h}, \mathbf{a}, r, \mathbf{o} \rangle \sim \mathcal{D}} \left[\left(y - \sum_i^n Q_i^\theta(h_i, a_i) \right)^2 \right], \text{ where } y = r + \gamma \sum_i^n \max_{a'_i} Q_i^{\theta^-}(h'_i, a'_i) \quad (4.2)$$

where h'_i is $h_i a_i o_i$ for h_i taken from $\mathbf{h}$, a_i from $\mathbf{a}$, and o_i from $\mathbf{o}$.

Since the loss uses the sum of the agent’s Q-functions, the result is a set of Q-values whose sum approximates the joint Q-function. This approximation will be lossless in extreme cases (e.g., full independence) but it may be close or still permit the agents to select the best actions even if the approximation is poor. The approach is also scalable since the max used to create the target value y is not over all agent actions as in the centralized case ($\mathbb{A}$), but done separately for each agent in Equation 4.2. Because action selection is over these local Q-values, the policy (and the resulting Q-function) is decentralized.

A simple version of VDN is given in Algorithm 7. The approach is very similar to standard DRQN [Hausknecht and Stone, 2015] as well as multi-agent DTE extensions [Omidshafiei et al., 2017, Tampuu et al., 2017] which are discussed in Sections 1.3.1 and 3.2. The key difference is the calculation of the loss on Line 20, which approximates the joint Q-value by using the sum of the individual Q-values.

In more detail, the Q_i are estimated using RNNs for each agent parameterized by θ_i , along with using a target network, θ_i^- , and a replay buffer, $\mathcal{D}$. First, episodes are generated by interacting with the environment (e.g., ϵ -greedy exploration), which are added to the replay buffer and indexed by episode number and time step ($\mathcal{D}^e(t)$). This buffer typically has a fixed size and new data replaces old data when the buffer is full. Episodes can then be sampled from the replay buffer (either single episodes or as a minibatch of episodes). In order to have the correct internal state, RNNs are trained sequentially. Training is done from the beginning of the episode until the end, calculating the internal state and the loss at each step and updating each θ_i using gradient descent. The target

²The paper also uses dueling [Wang et al., 2016] and investigates forms of weight sharing and communication but I view these as orthogonal to the main value decomposition contribution.

networks θ_i^- are updated periodically (every C episodes in the code) by copying the parameters from θ .

Algorithm 7 A version of value decomposition networks (VDN) (finite-horizon)

```

1: set  $\alpha$ ,  $\epsilon$ , and  $C$  (learning rate, exploration, and target update frequency)
2: Initialize network parameters  $\theta_i$  for each  $Q_i$  (denoted  $Q_i^\theta$ )
3: for all  $i$ ,  $\theta_i^- \leftarrow \theta_i$ 
4:  $\mathcal{D} \leftarrow \emptyset$ 
5:  $e \leftarrow 1$  {episode index}
6: for all episodes do
7:   for all  $h_i \leftarrow \emptyset$  {initial history is empty}
8:   for  $t = 1$  to  $\mathcal{H}$  do
9:     for all  $i$ , take  $a_i$  at  $h_i$  from  $Q_i^\theta(h_i, \cdot)$  with exploration (e.g.,  $\epsilon$ -greedy)
10:    See joint reward  $r_t$ , and observations  $\mathbf{o}_t$ 
11:    append  $\mathbf{a}, \mathbf{o}, r$  to  $\mathcal{D}^e$ 
12:    for all  $h_i \leftarrow h_i a_i o_i$  {update RNN state of the network}
13:  end for
14:  sample an episode from  $\mathcal{D}$ 
15:  for  $t = 1$  to  $\mathcal{H}$  do
16:    for all  $i$ ,  $h_i \leftarrow \emptyset$ 
17:     $\mathbf{a}, \mathbf{o}, r \leftarrow \mathcal{D}^e(t)$ 
18:    for all  $i$ ,  $h'_i \leftarrow h_i a_i o_i$ 
19:     $y = r + \gamma \sum_i \max_{a'_i} Q_i^{\theta^-}(h'_i, a'_i)$ 
20:    for all  $i$ , do gradient descent on  $\theta_i$  with learning rate  $\alpha$  and loss  $(y - \sum_i Q_i^\theta(h_i, a_i))^2$ 
21:    for all  $i$ ,  $h_i \leftarrow h'_i$ 
22:  end for
23:  if  $e \bmod C = 0$  then
24:    for all  $i$ ,  $\theta_i^- \leftarrow \theta_i$ 
25:  end if
26:   $e \leftarrow e + 1$ 
27: end for
28: return all  $Q_i$ 

```

As with the other cases, while the algorithm is presented for the finite-horizon case (for simplicity), it can easily be extended to the episodic and infinite-horizon cases by including terminal states or removing the loop over episodes.

After training, each agent keeps its own recurrent network, which can output the Q-values for that agent. The agent can then select an action by argmaxing over those Q-values: $\pi_i(h_i) = \operatorname{argmax}_{a_i} Q_i(h_i, a_i)$.

VDN is often used as a baseline but by using a simple sum to combine Q-functions, it can perform poorly compared to more sophisticated methods.

4.2.2 QMIX

QMIX [Rashid et al., 2018, 2020b] extends the value factorization idea of VDN to allow more general decompositions. In particular, instead of assuming the joint Q-function, $Q(\mathbf{h}, \mathbf{a})$, factors into

a sum of local Q-values, $Q_i(h_i, a_i)$, QMIX assumes the joint Q-function is a monotonic function of the individual Q-functions. This monotonic assumption means increases in local Q-functions lead to increases in the joint Q-function. The sum used in VDN is also a monotonic function but QMIX allows more general (possibly nonlinear) monotonic functions to be learned.

Specifically, QMIX assumes the following approximate factorization of the Q-function:

$$\mathbf{Q}(\mathbf{h}, \mathbf{a}) \approx f_{\text{mono}}(Q_1(h_1, a_1), \dots, Q_n(h_n, a_n)) \quad (4.3)$$

This monotonic assumption means the argmax over the local Q-functions is also an argmax over the joint Q-function. This property allows agents to choose actions from their local Q-functions instead of having to choose them from a joint Q-function—ensuring the choice would be the same in both cases. Like in VDN, it also makes the training more efficient since calculating the argmax in the update Equation 4.5 is linear in the number of agents rather than exponential.

Son et al. [2019] formalized this property as the Individual-Global-Max (IGM) principal. IGM states that the argmax over the joint Q-function is the same as argmaxing over each agent’s individual Q-function. The definition below is for a particular history but it should ideally hold for all histories (or at least the ones visited by the policy).

Definition 1 (Individual-Global-Max (IGM) [Son et al., 2019]) *For a joint action-value function $\mathbf{Q}(\mathbf{h}, \mathbf{a})$, where $\mathbf{h} = \langle h_1, \dots, h_n \rangle$ is a joint action-observation history, if there exist individual functions $[Q_i]$, such that:*

$$\arg\max_{\mathbf{a}} \mathbf{Q}(\mathbf{h}, \mathbf{a}) = \left(\begin{array}{c} \arg\max_{a_1} Q_1(h_1, a_1) \\ \vdots \\ \arg\max_{a_n} Q_n(h_n, a_n) \end{array} \right), \quad (4.4)$$

then, $[Q_i]$ satisfy IGM for $\mathbf{Q}$ at $\mathbf{h}$.

A simplified version of the QMIX architecture is shown in Figure 4.2. Like VDN, each agent has an RNN that takes in the current observation and can output Q-values for the updated history over all actions.³ A particular action is then chosen (e.g., using ϵ -greedy action selection) using each Q_i and each $Q_i(h_i, a_i)$ is fed into the mixing network. The mixing network is made to be monotonic by restricting the weights (but not the bias terms) to be non-negative. The mixing network can also receive the state to potentially improve performance. In practice, the state is given as input to hypernetworks [Ha et al., 2017] which generate the weights for the layers of the mixing network. Because the output depends on the state, it is $\mathbf{Q}(\mathbf{h}, s, \mathbf{a})$, and is often called $\mathbf{Q}_{\text{tot}}$.

The network is trained end-to-end, like VDN, by calculating a loss between $\mathbf{Q}(\mathbf{h}, s, \mathbf{a})$ and the joint return. The corresponding loss is:

$$\mathcal{L}(\theta) = \mathbb{E}_{\langle \mathbf{h}, s, \mathbf{a}, r, \mathbf{o}, s' \rangle \sim \mathcal{D}} \left[(y - \mathbf{Q}^\theta(\mathbf{h}, s, \mathbf{a}))^2 \right], \text{ where } y = r + \gamma \mathbf{Q}^{\theta^-}(\mathbf{h}', s', \tilde{\mathbf{a}}'),$$

$$\text{and } \tilde{\mathbf{a}}' = \langle \arg\max_{a'_1} Q_1(h'_1, a'_1), \dots, \arg\max_{a'_n} Q_n(h'_n, a'_n) \rangle \quad (4.5)$$

where the value on the next time step, $\mathbf{Q}^{\theta^-}(\mathbf{h}', s', \tilde{\mathbf{a}}')$, is gotten by inputting the argmax over each agent’s local Q-value using $\mathbf{h}' = \langle h'_1, \dots, h'_n \rangle$ as well as the corresponding state s' that is sampled from the buffer at $\mathbf{h}'$.

³QMIX typically also inputs the previous action but this is omitted here for clarity and consistency.

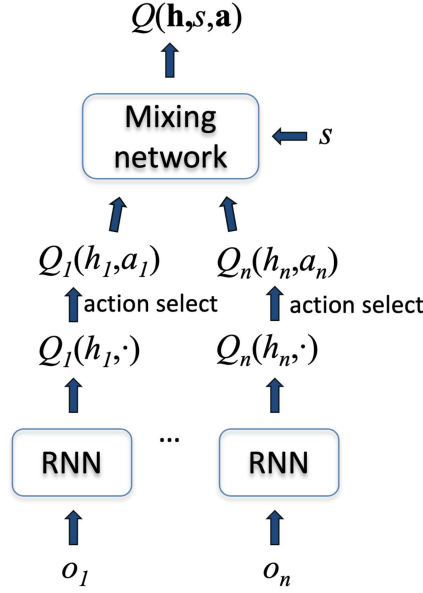


Figure 4.2: QMIX diagram

The mixing network is only needed during training. Agents can retain their RNNs (which are just DRQNs) for selecting actions during decentralized execution just like VDN.

Again, such a factorization may not be possible in all problems so QMIX may not be able to approximate the true joint Q-function in all problems accurately. In particular, as discussed in the paper, this factorization should fail when agents' action choices depend on (at least some) other agents' actions at the same time step.

4.2.3 Weighted QMIX

Weighted QMIX extends QMIX in an attempt to improve its expressiveness [Rashid et al., 2020a]. The main idea is to weigh the actions at different histories differently so the Q-values for some actions can be accurately represented but other action values can be less accurate. For instance, if you knew the optimal policy, you could set the weight highest for the optimal action in each history. You don't need to accurately represent the other action values but you do need to make sure they are lower than the optimal action (so the policy will choose the optimal action!).

Specifically, weighted QMIX uses weights $w(\mathbf{h}, \mathbf{a}) \in (0, 1]$ to adjust the importance of joint history-action pairs in the loss (as formalized below).⁴ It turns out that there always exists some weighing that will allow the optimal policy to be recovered by independently argmaxing over each agent's Q_i . An idealized form of the algorithm could also converge in the fully observable case but this proof doesn't extend to the partially observable case or when approximations are used (i.e., the deep RL case).

The method has two main components, a QMIX network where the output is weighted, and an approximation of the optimal Q-value called $\hat{Q}^*$. The first network takes the output of QMIX (as

⁴The paper discusses much of the method and theory in terms of the fully observable case but I consider the partially observable case here. In the partially observable case, the weights can take the history or the history and the state: $w(h, s, a)$.

seen in Figure 4.2) and weighs it as seen in Equation 4.6. For easier differentiation with the other Q network, I call the output of the first (QMIX-style) network $\mathbf{Q}_{tot}$. This network is exactly the same as the one in QMIX but the output is weighted in the loss to prioritize different actions at different histories. The $\hat{\mathbf{Q}}^*$ network is similar to the one in QMIX but it does not limit the weights to be non-negative and doesn't use a hypernetwork (just directly inputting the state into the mixing network). These networks don't share parameters and are trained using the losses given in Equation 4.6. The same target (y) is used in both cases, which now uses the unconstrained output $\hat{\mathbf{Q}}^*$. The argmax is done the same way as in QMIX, where the actions are chosen using each agent's Q_i . As a result, the argmax is tractable but a potentially more accurate value of those actions can be provided from $\hat{\mathbf{Q}}^*$ rather than $\mathbf{Q}_{tot}$.

In particular, the losses uses are:

$$\begin{aligned}\mathcal{L}(\theta_{cent}) &= \mathbb{E}_{\langle \mathbf{h}, s, \mathbf{a}, r, \mathbf{o}, s' \rangle \sim \mathcal{D}} \left[(y - \hat{\mathbf{Q}}^*(\mathbf{h}, s, \mathbf{a}))^2 \right], \\ \mathcal{L}(\theta_{tot}) &= \mathbb{E}_{\langle \mathbf{h}, s, \mathbf{a}, r, \mathbf{o}, s' \rangle \sim \mathcal{D}} \left[w(\mathbf{h}, \mathbf{a}) (y - \mathbf{Q}_{tot}(\mathbf{h}, s, \mathbf{a}))^2 \right], \\ \text{where } y &= r + \gamma \hat{\mathbf{Q}}^*(\mathbf{h}', s', \tilde{\mathbf{a}}'), \text{ and } \tilde{\mathbf{a}}' = \langle \underset{a'_1}{\operatorname{argmax}} Q_1(h'_1, a'_1), \dots, \underset{a'_n}{\operatorname{argmax}} Q_n(h'_n, a'_n) \rangle\end{aligned}\tag{4.6}$$

Two different weighting functions are considered. Centrally-Weighted QMIX (CW) uses:

$$w(\mathbf{h}, \mathbf{a}) = \begin{cases} 1 & \text{if } y > \hat{\mathbf{Q}}^*(\mathbf{h}, s, \tilde{\mathbf{a}}^*) \text{ or } \mathbf{a} = \tilde{\mathbf{a}}^* \\ \alpha & \text{otherwise} \end{cases}$$

which is an approximation of the optimal action $\tilde{\mathbf{a}}^*$ using the individual argmax as in Equation 4.6 and an approximation of the value function, $\hat{\mathbf{Q}}^*$, using the unconstrained network. The weight is set to 1 when the action is already known to be optimal ($\mathbf{a} = \tilde{\mathbf{a}}^*$) or when the action has a higher value than the current best action. Optimistically-Weighted QMIX (OW) uses:

$$w(\mathbf{h}, \mathbf{a}) = \begin{cases} 1 & \text{if } y > \mathbf{Q}_{tot}(\mathbf{h}, s, \mathbf{a}) \\ \alpha & \text{otherwise} \end{cases}$$

which just sets the weight to 1 when the current estimate using $\mathbf{Q}_{tot}$ is less than the target value, which uses $\hat{\mathbf{Q}}^*$. This inequality suggests error in $\mathbf{Q}_{tot}$ and $\mathbf{a}$ could (optimistically) be optimal at $\mathbf{h}$. α is a hyperparameter that can be set during training.

After training, the Q_i from the constrained network (outputting $\mathbf{Q}_{tot}$) can be used for decentralized action selection (just like in QMIX). Therefore, $\hat{\mathbf{Q}}^*$ is only used to help guide the learning so better (hopefully, optimal) actions will have higher values in each agent's Q_i . While weighted QMIX can outperform QMIX in some cases, it isn't as widely used as more recent methods (e.g., QPLEX [Wang et al., 2021a] and MAPPO [Yu et al., 2022]).

4.2.4 QTRAN

QTRAN [Son et al., 2019] provides a different way to factor the joint Q-function into individual Q-functions. The idea generalizes VDN but the intuition is that the sum does not have to equal the true Q-function but some *transformed* Q-function $\mathbf{Q}'$. A separate V term⁵ can then be used to

⁵While the paper calls this a state value, the value takes histories as input, not states.

account for the error between the transformed $\mathbf{Q}'$ and the true $\mathbf{Q}$. The approach cannot represent all IGM-able functions but it is more general than VDN.⁶ QPLEX (which I talk about below) is more general and the relationship to QMIX and weighted QMIX is unclear.

Specifically, QTRAN uses Equation 4.7 as the basis for their architecture and losses.

$$\sum_i Q_i(h_i, a_i) - \mathbf{Q}(\mathbf{h}, \mathbf{a}) + \mathbf{V}(\mathbf{h}) = \begin{cases} 0 & \mathbf{a} = \tilde{\mathbf{a}} \\ \geq 0 & \mathbf{a} \neq \tilde{\mathbf{a}} \end{cases} \quad (4.7)$$

where

$$\mathbf{V}(\mathbf{h}) = \max_{\mathbf{a}} \mathbf{Q}(\mathbf{h}, \mathbf{a}) - \sum_i Q_i(h_i, \tilde{a}_i),$$

$$\tilde{a}_i = \operatorname{argmax}_{a_i} Q_i(h_i, a_i), \quad \text{and} \quad \tilde{\mathbf{a}} = \langle \tilde{a}_1, \dots, \tilde{a}_n \rangle$$

Here, $\mathbf{V}(\mathbf{h})$ has a particular form where it is the difference between the (centralized) max over the joint Q-function, $\mathbf{Q}$, and the sum of the maxes over the individual Q-functions, Q_i . Equation 4.7 is constrained to be 0 when the actions are the argmax over the individual Q-functions, $\tilde{\mathbf{a}}$, and greater than or equal to 0 for other actions. If Equation 4.7 holds, the Q_i satisfy IGM for the $\mathbf{Q}$ but it isn't clear how general it is. That is, unlike QPLEX, the proof does not show that all functions that satisfy IGM can be represented in this form.

Instead of enforcing a network structure to get IGM like VDN and QMIX, QTRAN does so by using losses based on Equation 4.7. The architecture outputs $\mathbf{Q}'(\mathbf{h}, \mathbf{a}) = \sum_i Q_i(h_i, a_i)$, like VDN, as well as an unconstrained $\mathbf{Q}(\mathbf{h}, \mathbf{a})$ and $\mathbf{V}(\mathbf{h})$. As shown below, there is the standard TD loss for learning the true joint Q-function, $\mathbf{Q}$, a loss for learning the transformed Q-function, $\mathbf{Q}'$, and value offset, $\mathbf{V}$, for the given $\mathbf{Q}$ when the actions are currently the maximizing ones, $\tilde{\mathbf{a}}$, in $\mathcal{L}_{opt}(\theta)$ and for other actions in $\mathcal{L}_{nopt}(\theta)$. The notation $\bar{\mathbf{Q}}$ is used to note that $\mathbf{Q}$ isn't trained from $\mathcal{L}_{opt}(\theta)$ and $\mathcal{L}_{nopt}(\theta)$.

$$\begin{aligned} \mathcal{L}_{td}(\theta) &= \mathbb{E}_{\langle \mathbf{h}, \mathbf{a}, r, \mathbf{o} \rangle \sim \mathcal{D}} \left[(y - \mathbf{Q}(\mathbf{h}, \mathbf{a}))^2 \right], \\ \mathcal{L}_{opt}(\theta) &= \mathbb{E}_{\langle \mathbf{h}, \mathbf{a}, r, \mathbf{o} \rangle \sim \mathcal{D}} \left[(\mathbf{Q}'(\mathbf{h}, \tilde{\mathbf{a}}) - \bar{\mathbf{Q}}(\mathbf{h}, \tilde{\mathbf{a}}) + \mathbf{V}(\mathbf{h}))^2 \right], \\ \mathcal{L}_{nopt}(\theta) &= \mathbb{E}_{\langle \mathbf{h}, \mathbf{a}, r, \mathbf{o} \rangle \sim \mathcal{D}} \left[\left(\min [\mathbf{Q}'(\mathbf{h}, \mathbf{a}) - \bar{\mathbf{Q}}(\mathbf{h}, \mathbf{a}) + \mathbf{V}(\mathbf{h}), 0] \right)^2 \right], \\ \text{where } y &= r + \gamma \mathbf{Q}_{\theta-}(\mathbf{h}', \tilde{\mathbf{a}}'), \text{ and } \tilde{\mathbf{a}}' = \langle \operatorname{argmax}_{a'_1} Q_1(h'_1, a'_1), \dots, \operatorname{argmax}_{a'_n} Q_n(h'_n, a'_n) \rangle \end{aligned} \quad (4.8)$$

The losses above are used for the standard form of QTRAN called QTRAN-base. An alternate form, called QTRAN-alt, replaces $\mathcal{L}(\theta_{nopt})$ with the average counterfactual loss below, forcing the value to be 0 for some action rather than relying on the inequality constraint above.

$$\mathcal{L}(\theta_{nopt-min}) = \mathbb{E}_{\langle \mathbf{h}, \mathbf{a}, r, \mathbf{o} \rangle \sim \mathcal{D}} \left[\frac{1}{n} \sum_i \left(\min [\mathbf{Q}'(\mathbf{h}, a_i, \mathbf{a}_{-i}) - \bar{\mathbf{Q}}(\mathbf{h}, a_i, \mathbf{a}_{-i}) + \mathbf{V}(\mathbf{h}), 0] \right)^2 \right],$$

Using this condition also satisfies IGM.

The versions of QTRAN (QTRAN-base and QTRAN-alt) can outperform VDN and QMIX on some domains but they are often outperformed by more recent methods.

⁶While some have taken the text of the paper to show that QTRAN can represent all IGM-able functions, the proofs only show that the method satisfies IGM (not that any IGM-able function can be represented by the method).

4.2.5 QPLEX

QPlex [Wang et al., 2021a] further extends the value factorization idea to (provably) include more general decentralized policies. In particular, QPLEX can potentially learn any set of Q-functions that satisfy the IGM principle.

First, QPLEX extends the IGM principle to an advantage-based case. They define joint values and advantages from the joint Q-function: $\mathbf{V}(\mathbf{h}) = \max_{\mathbf{a}} \mathbf{Q}(\mathbf{h}, \mathbf{a})$, and $\mathbf{A}(\mathbf{h}, \mathbf{a}) = \mathbf{Q}(\mathbf{h}, \mathbf{a}) - \mathbf{V}(\mathbf{h})$ as well as individual values and advantages from each agent’s individual Q-function: $V_i(h_i) = \max_{a_i} Q_i(h_i, a_i)$, and $A_i(h_i, a_i) = Q_i(h_i, a_i) - V_i(h_i)$. Note that this notion of advantage is a bit different than the typical notion (such as [Wang et al., 2016]) since the values and advantages are generated from the Q-values, rather than the other way around. As a result, the advantages will have the property that they will be 0 for optimal actions and negative otherwise. That is, $A_i(h_i, a_i^*) = 0$ (since $Q_i(h_i, a_i^*) - \max_{a_i} Q_i(h_i, a_i) = 0$) and for some non-optimal action $a_i^\dagger$, $A_i(h_i, a_i^\dagger) < 0$ (since $Q_i(h_i, a_i^\dagger) < \max_{a_i} Q_i(h_i, a_i)$). The same holds true for joint advantages.

Advantage-based IGM extends IGM to the advantage case and states that the argmax over the joint advantage function is the same as argmaxing over each agent’s individual advantage function:

Definition 2 (Advantage-based IGM) Wang et al. [2021a)] *For a joint action-value function, if there exist individual functions $[Q_i]$, such that:*

$$\operatorname{argmax}_{\mathbf{a}} \mathbf{A}(\mathbf{h}, \mathbf{a}) = \begin{pmatrix} \operatorname{argmax}_{a_1} A_1(h_1, a_1) \\ \vdots \\ \operatorname{argmax}_{a_n} A_n(h_n, a_n) \end{pmatrix}. \quad (4.9)$$

where $\mathbf{A}(\mathbf{h}, \mathbf{a})$ and A_i are defined above, then $[Q_i]$ satisfy advantage-based IGM for $\mathbf{Q}$ at $\mathbf{h}$.

That is, advantage-based IGM is equivalent to the Q-based IGM in Definition 1.

QPlex uses advantage-based IGM to extend QMIX and QTRAN to represent the full IGM function class. The architecture is shown in Figure 4.3. Like QMIX (and VDN and QTRAN), QPLEX first takes as input each agent’s observation o_i (and previous action a_i^{t-1} but that is not included here) and outputs the individual Q-values, $Q_i(h_i, a_i)$, for all a_i using a DRQN-style network. After an action is selected (e.g., using ϵ -greedy exploration), the value and advantage are extracted from the Q-values for each agent as $V_i(h_i) = \max_{a_i} Q_i(h_i, a_i)$ and $A_i(h_i, a_i) = Q_i(h_i, a_i) - V_i(h_i)$. Next, the transformation networks generate $V_i(h_i, s)$ and $A_i(h_i, s, a_i)$ for each agent with the given $V_i(h_i)$ and $A_i(h_i, a_i)$ along with s as $V_i(h_i, s) = w_i(s)V_i(h_i) + b_i(s)$ and $A_i(h_i, s, a_i) = w_i(s)A_i(h_i, a_i)$. Finally, the joint Q-value is output based on the output of each agent’s transformation network and the state as $\mathbf{Q}(\mathbf{h}, s, \mathbf{a}) = \sum_i V_i(h_i, s) + \lambda_i(s, \mathbf{a})A_i(h_i, s, a_i)$.⁷ While there aren’t constraints on $\mathbf{V}$ due to IGM (because it doesn’t include the actions), QPLEX uses a sum to combine the local $\mathbf{V}$ ’s. All the weights, w_i and λ_i (but not necessarily the biases b_i) are positive to ensure the advantage is 0 for the max action and negative otherwise in order to satisfy the advantage IGM principle. That is, instead of using monotonicity like VDN and QMIX, QPLEX achieves IGM by constraining the max action to have a value of 0 for the joint and individual advantages and the advantages for other actions is negative. This constraint is exactly what IGM says—the argmax of the joint $\mathbf{Q}$ (or $\mathbf{A}$) is the argmax of the individual Q_i (or A_i). The architecture is trained using the RL loss, just like QMIX in Equation 4.5.

⁷Attention is used to train the λ weights more efficiently.

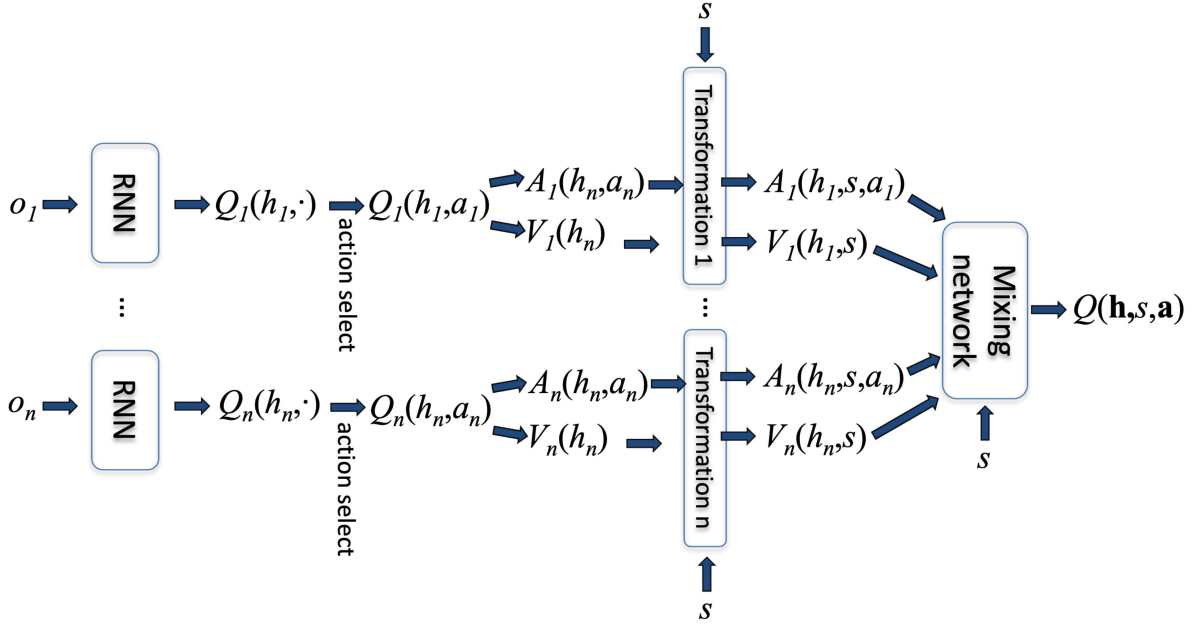


Figure 4.3: QPLEX diagram

As a result of the advantage-based architecture, QPLEX is more general than previous methods with the capacity to represent all Q-functions that satisfy IGM. QPLEX also performs well, often outperforming other value factorization methods and it is currently one of the best-performing CTDE methods.

4.2.6 The use of state in factorization methods

Many of the current methods use inconsistent notation about the inclusion of state (except QMIX). As a result, the theory is often developed without considering the state input. It turns out that, unlike the policy gradient case discussed in Section 4.3.6, using the state doesn't introduce bias and is thus theoretically sound [Marchesini et al., 2025]. The intuition is that the state is additional information in these cases. It doesn't replace the history, but potentially augments it (similar to a history-state critic in Section 4.3.6). Furthermore, actions are not chosen based on the state but only the local, history-dependent Q-values, Q_i . Nevertheless, in implementations of algorithms such as QPLEX and weighted QMIX, the weights only take the state as input and not the history. This replacement of history with state in these contexts limits the representational power of the weights in partially observable problems. As a result, using the state as input in QPLEX (instead of the joint history) prevents it from being able to represent the full IGM class of functions. It still may be beneficial to use the state instead of the joint history but exploring what additional information to use and in what way is an interesting open question.

4.2.7 Other value function factorization methods

Qatten [Yang et al., 2020] extends QMIX (and weighted QMIX) to add structure to the mixing network, where the structure is inspired by a linear approximation of the Q-function and one set of



Figure 4.4: Decentralized critics (a) vs. a centralized critic (b).

weights is learned using attention. A wide range of other approaches have been developed but are not included in this introduction to the area.

4.3 Centralized critic methods

Concurrently, Multi-Agent Deep Deterministic Policy Gradient (MADDPG) [Lowe et al., 2017] and COunterfactual Multi-Agent policy gradient (COMA) [Foerster et al., 2018b] popularized the use of centralized critics in MARL. I will first discuss the general class of algorithms that MADDPG and COMA represent—multi-agent actor-critic methods with centralized critics—and then give the details of MADDPG and COMA. I will then describe the very popular PPO-based extension, MAPPO [Yu et al., 2022]. Finally, I will discuss the (incorrect) use of state in centralized critics as well as the theoretical and practical tradeoffs in the various types of critics.

4.3.1 Preliminaries

Single-agent policy gradient and actor-critic methods have also been extended to the CTDE case. In the multi-agent case, there is one actor per agent and either one critic per agent (as shown in 4.4(a) and discussed in Section 3.3) or a shared, centralized critic (as shown in 4.4(b) and discussed below). The centralized critic can be used during CTDE but then each agent can act in a decentralized manner by using its actor. The basic motivation is to leverage the centralized information that is available during training and potentially combat nonstationarity that can happen in decentralized training. All the policy gradient approaches can generally scale to larger action spaces and have stronger (local) convergence guarantees than the value-based counterparts above.

Policy gradient methods use continuous-action or stochastic policies. Unless stated otherwise, I will assume a stochastic policy for each agent parameterized by ψ_i , where $\pi_i^{\psi_i}(a_i|h_i)$ represents the probability agent i will choose action a_i given the history h_i and parameters, ψ_i , $\Pr(a_i|h_i, \psi_i)$. Like the value-based methods, the CTDE policy gradient methods also assume algorithms receive agent actions, $\mathbf{a}$, the resulting observations, $\mathbf{o}$, and the joint reward, r at each time step.

4.3.2 A basic centralized critic approach

In the simplest form, we can consider a class of centralized critic methods where the joint history critic, which I denote as $\hat{Q}(\mathbf{h}, \mathbf{a})$, estimates $Q^\pi(\mathbf{h}, \mathbf{a})$ with parameters θ . $\hat{Q}(\mathbf{h}, \mathbf{a})$ is then used to update each decentralized policy π_i . We call this approach independent actor with centralized critic (IACC) and it is described in Algorithm 8. Learning is over a set of (on-policy) episodes and the histories are initialized to be empty. Agent i 's history at time step t is denoted $h_{i,t}$, while the set of histories for all agents is denoted $\mathbf{h}_t$. All agents choose an initial action from their

policies $a_{i,0} \sim \pi_i(a_i|h_{i,0})$ and then the learning takes place over a fixed horizon of $\mathcal{H}$. At each step, the agents take the corresponding current action $\mathbf{a}_t$ and see observations $\mathbf{o}_t$ and the algorithm receives the joint reward r_t . The histories for the next step are updated to include the action taken and observation seen, $h_{i,t}a_{i,t}o_{i,t}$, and actions are sampled for the next step, $a_{i,t+1} \sim \pi_i(a_i|h_{i,t+1})$. Then, the TD error can be computed with the current Q-values, and it is used to update the actors and critic. Each agent's actor is updated using the TD error from the joint Q-function (moving in the direction that improves the joint Q-value). The centralized critic is updated using a standard on-policy gradient update. The action for the next step, $t + 1$, becomes the action for the current step, t , and the process continues until the end of the horizon and for each episode. The algorithm is written for the finite-horizon case but it can be adapted to the episodic or infinite-horizon case by including terminal states or removing the loop over episodes. Note that the histories make the value functions based on a particular time step as the history length provides the time step.

While a sample is used in the algorithm, the full gradient associated with each actor in IACC is represented as:

$$\nabla_{\psi_i} J = \mathbb{E}_{\langle \mathbf{h}, \mathbf{a} \rangle \sim \mathcal{D}} [\mathbf{Q}^\pi(\mathbf{h}, \mathbf{a}) \nabla_{\psi_i} \log \pi_i(a_i|h_i)] ,^8 \quad (4.10)$$

where the approximation of the joint Q-function, $\hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a})$, is used to update the policy parameters of agent i 's actor, $\pi_i(a_i|h_i)$. Histories (i.e., observations) and actions are sampled according to the on-policy (discounted) visitation probability. The objective, J , is to maximize expected (discounted) return starting from the initial state distribution b_0 as discussed in Section 1.2.

The critic is updated using the on-policy loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{\langle \mathbf{h}, \mathbf{a}, r, \mathbf{h}' \rangle \sim \mathcal{D}} \left[(y - \hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a}))^2 \right], \text{ where } y = r + \gamma \hat{\mathbf{Q}}(\mathbf{h}', \mathbf{a}')$$

We can extend this algorithm to the more commonly used advantage actor-critic (A2C) case, as shown in Algorithm 9. The advantage is defined as $\mathbf{A}(\mathbf{h}, \mathbf{a}) = \mathbf{Q}(\mathbf{h}, \mathbf{a}) - \mathbf{V}(\mathbf{h})$, representing the difference between the Q-value for a given action and the V-value at that history.⁹ Because $\mathbf{V}(\mathbf{h})$ doesn't depend on the action, it is called a *baseline* and becomes a constant from the perspective of the policy gradient. As a result, using $\mathbf{Q}(\mathbf{h}, \mathbf{a}) - \mathbf{V}(\mathbf{h})$ in place of $\mathbf{Q}(\mathbf{h}, \mathbf{a})$ doesn't change the convergence properties of the method. Nevertheless, using a baseline (e.g., subtracting $\mathbf{V}(\mathbf{h})$) can reduce the variance of the estimate and improve performance in practice. In the algorithm, $\mathbf{A}(\mathbf{h}, \mathbf{a})$ isn't explicitly stored since the TD error is used as an approximation for the advantage— $\hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a})$ is approximated with $r_t + \gamma \hat{\mathbf{V}}(\mathbf{h}_{t+1})$, which is true in expectation. δ_t can be thought of as a sample of $\mathbf{Q}(\mathbf{h}_t, \mathbf{a}_t) - \hat{\mathbf{V}}(\mathbf{h}_t) = \hat{\mathbf{A}}(\mathbf{h}_t, \mathbf{a}_t)$.

While the critic is called *centralized* because it uses centralized information, it estimates the *joint* value function of the decentralized policies. That is, the centralized critic estimates the value of the current set of decentralized policies (not a centralized policy) [Lyu et al., 2023]. This is the correct thing to do since the critic's job is to evaluate the policies, which are decentralized in this case.

With appropriate assumptions (e.g., on exploration, learning rate, and function approximation), IACC (and IA2CC) will converge to a local optimum [Peshkin et al., 2000, Lyu et al., 2023, 2024]. These results put policy gradient methods on solid theoretical ground and I discuss details of the various methods as well as some theoretical shortcomings.

⁸Again, for simplicity discounting is removed but the true gradient would include it [Nota and Thomas, 2020].

⁹Note that this advantage definition is the standard one that, unlike QPLEX, allows positive and nonpositive advantages.

Algorithm 8 Independent Actor Centralized Critic (IACC) (finite-horizon)

```

1: Initialize individual actor models  $\pi_i(a_i|h_i)$ , parameterized by  $\psi_i$ 
2: Initialize centralized critic model  $\hat{Q}(\mathbf{h}, \mathbf{a})$ , parameterized by  $\theta$ 
3: for all episodes do
4:    $h_{i,0} \leftarrow \emptyset$  { Empty initial history }
5:   Denote  $\mathbf{h}_t$  as  $\langle h_{1,0}, \dots, h_{n,0} \rangle$  { Notation for joint variables }
6:   for all  $i$ , choose  $a_{i,0}$  at  $h_{i,0}$  from  $\pi_i(a_i|h_{i,0})$ 
7:   Store  $\mathbf{a}_t$  as  $\langle a_{1,0}, \dots, a_{n,0} \rangle$ 
8:   for  $t = 0$  to  $\mathcal{H} - 1$  do
9:     Take joint action  $\mathbf{a}_t$ , see joint reward  $r_t$ , and observations  $\mathbf{o}_t$ 
10:    for all  $i$ ,  $h_{i,t+1} \leftarrow h_{i,t} a_{i,t} o_{i,t}$  { Append new action and obs to previous history }
11:    for all  $i$ , choose  $a_{i,t+1}$  at  $h_{i,t+1}$  from  $\pi_i(a_i|h_{i,t+1})$ 
12:    Store  $\mathbf{a}_{t+1}$  as  $\langle a_{1,t+1}, \dots, a_{n,t+1} \rangle$ 
13:     $\delta_t \leftarrow r_t + \gamma \hat{Q}(\mathbf{h}_{t+1}, \mathbf{a}_{t+1}) - \hat{Q}(\mathbf{h}_t, \mathbf{a}_t)$  { Compute centralized TD error }
14:    Compute critic gradient estimate:  $\delta_t \nabla_{\theta} \hat{Q}(\mathbf{h}_t, \mathbf{a}_t)$ 
15:    Update critic parameters  $\theta$  using gradient estimate (e.g.,  $\theta \leftarrow \theta + \beta \delta_t \nabla_{\theta} \hat{Q}(\mathbf{h}_t, \mathbf{a}_t)$  for learning rate  $\beta$ )
16:    for each agent  $i$  do
17:      Compute actor gradient estimate:  $\gamma^t \hat{Q}(\mathbf{h}_t, \mathbf{a}_t) \nabla_{\psi_i} \log \pi_i(a_{i,t}|h_{i,t})$ 
18:      Update actor parameters  $\psi_i$  using gradient estimate (e.g.,  $\psi_i \leftarrow \psi_i + \alpha \gamma^t \hat{Q}(\mathbf{h}, \mathbf{a}) \nabla_{\psi_i} \log \pi_i(a_{i,t}|h_{i,t})$  for learning rate  $\alpha$ )
19:    end for
20:  end for
21: end for

```

Algorithm 9 Independent Advantage Actor Centralized Critic (IA2CC) (finite-horizon)

```

1: Initialize individual actor models  $\pi_i(a_i|h_i)$ , parameterized by  $\psi_i$ 
2: Initialize centralized critic model  $\hat{V}(\mathbf{h})$ , parameterized by  $\theta$ 
3: for all episodes do
4:   for all  $i$ ,  $h_{i,0} \leftarrow \emptyset$  {Empty initial history}
5:   Denote  $\mathbf{h}_t$  as  $\langle h_{1,0}, \dots, h_{n,0} \rangle$  {Notation for joint variables}
6:   for all  $i$ , choose  $a_{i,0}$  at  $h_{i,0}$  from  $\pi_i(a_i|h_{i,0})$ 
7:   Store  $\mathbf{a}_t$  as  $\langle a_{1,0}, \dots, a_{n,0} \rangle$ 
8:   for  $t = 0$  to  $\mathcal{H} - 1$  do
9:     Take joint action  $\mathbf{a}_t$ , see joint reward  $r_t$ , and observations  $\mathbf{o}_t$ 
10:    for all  $i$ ,  $h_{i,t+1} \leftarrow h_{i,t} a_{i,t} o_{i,t}$  {Append new action and obs to previous history}
11:    for all  $i$ , choose  $a_{i,t+1}$  at  $h_{i,t+1}$  from  $\pi_i(a_i|h_{i,t+1})$ 
12:    Store  $\mathbf{a}_{t+1}$  as  $\langle a_{1,t+1}, \dots, a_{n,t+1} \rangle$ 
13:     $\delta_t \leftarrow r_t + \gamma \hat{V}(\mathbf{h}_{t+1}) - \hat{V}(\mathbf{h}_t)$  {Compute centralized advantage estimates (TD error)}
14:    Compute critic gradient estimate:  $\delta_t \nabla \hat{V}(\mathbf{h}_t)$ 
15:    Update critic parameters  $\theta$  using gradient estimate (e.g.,  $\theta \leftarrow \theta + \beta \gamma \delta_t \nabla \hat{V}(\mathbf{h}_t)$ )
16:    for each agent  $i$  do
17:      Compute actor gradient estimate:  $\gamma^t \delta_t \nabla \log \pi_i(a_{i,t}|h_{i,t})$ 
18:      Update actor parameters  $\psi_i$  using gradient estimate (e.g.,  $\psi_i \leftarrow \psi_i +$   

        $\alpha \gamma^t \delta_t \nabla \log \pi_i(a_{i,t}|h_{i,t})$ )
19:    end for
20:  end for
21: end for

```

This idea of learning a centralized critic to update decentralized actors is very general. It can be (and has been) used with different types of critics, actors, and updates. I present some of the most popular methods below.

4.3.3 MADDPG

Multi-Agent DDPG (MADDPG) [Lowe et al., 2017] is a centralized critic method that considers the more general case of possibly competitive agents as well as continuous actions. To deal with the different reward functions of different agents in the competitive case, separate centralized critics are learned for each agent. As I am only concerned with the cooperative case in this text, I assume a single shared critic among agents, do not consider learning policy models of the other agents (since these are assumed to be accessible in a cooperative CTDE setting), and do not consider ensembles of other agent policies to improve robustness. MADDPG is also an off-policy method, unlike the previous algorithms discussed, so it makes use of replay buffer similar to DQN-based approaches. Nevertheless, MADDPG for the cooperative case is very similar to Algorithm 8 with the main changes noted below.

To deal with continuous actions, MADDPG extends the continuous action single-agent deep actor-critic method DDPG [Lillicrap et al., 2016] to the multi-agent case. I denote the deterministic continuous-action policies for each agent as μ_i . In the cooperative case, MADDPG uses the following policy gradient:

$$\nabla_{\psi_i} J = \mathbb{E}_{x, \mathbf{a} \sim \mathcal{D}} \left[\nabla_{\psi_i} \mu_i(o_i) \nabla_{\mathbf{a}} \mathbf{Q}^{\pi}(x, \mathbf{a}) \mid_{a_i = \mu_i(o_i)} \right], \quad (4.11)$$

where x and $\mathbf{a}$ are sampled from the buffer with x discussed below and $\mathbf{a} = \langle a_1, \dots, a_n \rangle$. Since we can no longer sum over actions and weigh their value by their probability as is done in the stochastic policy case, we have to consider the change in the Q-value function evaluated at the chosen action a_i for the agent [Silver et al., 2014].

Note that MADDPG considers reactive policies that only map from the last observation to an action (as seen in μ_i). As noted in Section 1.2, the policy, μ_i , should choose actions based on histories, h_i , rather than single observations, o_i . The Q-value is from a centralized critic so the paper states x “could consist of the observations of all agents [...] however, we could also include additional state information if available.” More generally, x should be history representations for each agent and I discuss the use of states in more detail in 4.3.6. These issues are easily fixable and result in the following policy gradient (we can incorporate the state information later):

$$\nabla_{\psi_i} J = \mathbb{E}_{x, \mathbf{a} \sim \mathcal{D}} \left[\nabla_{\psi_i} \mu_i(h_i) \nabla_{\mathbf{a}} \mathbf{Q}^{\pi}(\mathbf{h}, \mathbf{a}) \mid_{a_i = \mu_i(h_i)} \right]. \quad (4.12)$$

Because MADDPG is off-policy, it maintains a replay buffer like DQN-based approaches along with ‘target’ copies of both the actor and critic networks, $\mu_{\psi_i^-}$ and Q_{θ^-} . The Q-update (for the history-based case) is then:

$$\mathcal{L}(\theta) = \mathbb{E}_{\langle \mathbf{h}, \mathbf{a}, r, \mathbf{h}' \rangle \sim \mathcal{D}} \left[(y - Q_{\theta}(\mathbf{h}, \mathbf{a}))^2 \right], \text{ where } y = r + \gamma Q_{\theta^-}(\mathbf{h}', \mathbf{a}') \mid_{a_i = \mu_i^-(h_i) \forall i \in \mathbb{I}} \quad (4.13)$$

MADDPG executes a stochastic exploration policy (e.g., by adding Gaussian noise to the current deterministic policy) while estimating (and optimizing) the value of the deterministic policy. This can be seen because while the action, $\mathbf{a}$, is sampled from the (behavior policy) dataset, the next action, $\mathbf{a}'$ is selected by the target policy, μ^- .

Algorithm 8 can be updated to incorporate these changes. Since the continuous-action policy is deterministic, (Gaussian) noise is added when selecting actions to aid in exploration. The approach is off-policy, so the experiences are first stored in a replay buffer (like DQN and other value-based methods in Section 4.2). Similarly, episodes are sampled from the replay buffer and target networks used (again, like DRQN-based approaches) for the actor and the critic.

MADDPG is no longer widely used but the ideas (such as the centralized critic) have been adopted extensively.

4.3.4 COMA

The main contributions of Counterfactual Multi-Agent Policy Gradients (COMA) were the introduction of the centralized critic along with a counterfactual baseline [Foerster et al., 2018b]. As mentioned above, baselines are common in policy gradient methods since they are high variance. The baseline value is subtracted from the Q-value and it can be anything that is not dependent on the agent’s action.

In the case of COMA, the motivation for the baseline is not only variance reduction but also better credit assignment by subtracting off the perceived contribution to the Q-value from the other agents. Specifically, it marginalizes out the agent actions from the Q-function to get an estimate of what the (counterfactual) Q-function would be while holding the other agent actions fixed. The result is an agent-specific advantage function that subtracts the baseline for agent i from the joint Q-function:

$$A_i(\mathbf{h}, \mathbf{a}) = Q(\mathbf{h}, \mathbf{a}) - \sum_{a'_i} \pi_i(a'_i | h_i) Q(\mathbf{h}, a'_i, \mathbf{a}_{-i})$$

This baseline no longer depends on agent i ’s action (in expectation) so it will not bias the gradient. Note that the COMA paper uses a state-based critic and advantage, $Q(s, \mathbf{a})$ and $A_i(s, \mathbf{a})$, but this is incorrect as I’ll discuss in Section 4.3.6.

Rather than have separate baseline networks for each agent, there is a single centralized critic that takes in the other agent actions, $\mathbf{a}_{-i}$, the joint observation, $\mathbf{o}$, an agent id, i , the proposed action a_i , and policy probabilities $\pi_i(a'_i | h_i)$, and outputs the advantage for that agent $A_i(\mathbf{h}, \mathbf{a})$ using the equation above. This network can also output the joint Q-values for all agent i ’s actions, $Q(\mathbf{h}, \cdot, \mathbf{a}_{-i})$, given the other agent actions, $\mathbf{a}_{-i}$, the joint observation, $\mathbf{o}$, and an agent id, i . Both of these values are needed for the algorithm.

The COMA algorithm can also be an extension of Algorithm 8. The critic update can be calculated in the same way (using the Q-values from the network described above) but the agent-specific advantage is incorporated in the actor update. That is, during the actor update (starting on line 16), the agent-advantage $A_i(\mathbf{h}, \mathbf{a})$ is calculated for the given agent and A_i is used in the actor gradient estimate instead of $\hat{Q}$: $\gamma^t A_i(\mathbf{h}_t, \mathbf{a}_t) \nabla_{\psi_i} \log \pi_i(a_{i,t} | h_{i,t})$.

While COMA has been very influential, it isn’t widely used since newer methods tend to outperform it.

4.3.5 MAPPO

Multi-Agent PPO (MAPPO) extends PPO [Schulman et al., 2017] to the centralized critic MARL case [Yu et al., 2022]. The motivation behind PPO is to adjust the magnitude of the policy up-

date to learn quickly without becoming unstable. PPO does this using a simple clipped loss that approximates the more complex trust region update [Schulman et al., 2015].

Like IA2CC (Algorithm 9), MAPPO uses an advantage-based update (but not the agent-specific counterfactual one used in COMA). In particular, instead of the loss being $\gamma^t \delta_t \log \pi_i(a_{i,t}|h_{i,t})$ as reflected on line 17, which is an approximation of $\gamma^t \mathbf{A}_t \log \pi_i(a_{i,t}|h_{i,t})$, the loss becomes:

$$\mathcal{L}_{clip}^{MAPPO}(\psi_i) = \min \left(r_{\psi_i,i} \mathbf{A}, \text{clip}(r_{\psi_i,i}, 1 - \epsilon, 1 + \epsilon) \mathbf{A} \right), \quad (4.14)$$

where $r_{\psi_i,i} = \frac{\pi_{\psi_i}(a_i|h_i)}{\pi_{\psi_{i,old}}(a_i|h_i)}$.¹⁰ Maximizing $\frac{\pi_{\psi_i}(a_i|h_i)}{\pi_{\psi_{i,old}}(a_i|h_i)} \mathbf{A}$ seeks to maximally improve the new policy, π_{ψ_i} , compared to the old policy, $\pi_{\psi_{i,old}}$, by reweighing the advantage using an importance sampling ratio (considering the advantage was calculated using the old policy).

It can be difficult to estimate this value from a small number of samples and it may result in too large of an update (resulting in parameters that perform worse). As a result, PPO also considered a term using a clipped ratio, limiting $r_{\psi_i,i}$ so it can't be too far above or below 1 (which would be the value when the new policy is equal to the old one). By minimizing over the unclipped and clipped values, PPO limits changes in the policy. Note the ratio is always positive but the advantage could be positive or negative. When the advantage is positive, the loss will be clipped if the ratio is too large, limiting how much more likely the action can be in the policy. Similarly, when the advantage is negative, the loss will be clipped when the ratio is near 0, limiting how much less likely the action can be in the policy. Like in IA2CC, $\mathbf{A}$ can still be approximated as $r_t + \gamma \hat{\mathbf{V}}(\mathbf{h}_{t+1}) - \hat{\mathbf{V}}(\mathbf{h}_t)$.

The critic loss for MAPPO also includes clipping and is given by:

$$\mathcal{L}^{MAPPO}(\theta) = \max \left[(\mathbf{V}(\mathbf{h}_t) - \hat{R}_t)^2, \left(\text{clip}(\mathbf{V}(\mathbf{h}), \mathbf{V}_{old}(\mathbf{h}) - \epsilon, \mathbf{V}_{old}(\mathbf{h}) + \epsilon) - \hat{R}_t \right)^2 \right] \quad (4.15)$$

where $\hat{R}$ are Monte Carlo returns starting from the current history (i.e., $\hat{R}_t = \sum_{j=t}^{\mathcal{H}} \gamma r_j$) and $\mathbf{V}_{old}$ represents the value function from the previous time step but TD can also be used and value clipping does not have to be used.

All agents use the same centralized critic and parameter sharing can also be used so all agents share the same actor network.

Independent PPO (IPPO) is a version of MAPPO where each agent uses its own local advantage rather than the joint advantage. As a result, the IPPO actor loss becomes:

$$\mathcal{L}_{clip}^{IPPO}(\psi_i) = \min \left(r_{\psi_i,i} A_i, \text{clip}(r_{\psi_i,i}, 1 - \epsilon, 1 + \epsilon) A_i \right), \quad (4.16)$$

with $r_{\psi_i,i} = \frac{\pi_{\psi_i}(a_i|h_i)}{\pi_{\psi_{i,old}}(a_i|h_i)}$ as before and $\hat{A}_i = r_t + \gamma \hat{V}_i(h_{i,t+1}) - \hat{V}_i(h_{i,t})$. The IPPO critic loss becomes:

$$\mathcal{L}^{IPPO}(\theta) = \max \left[(V_i(h_{i,t}) - \hat{R}_t)^2, \left(\text{clip}(V_i(h_{i,t}), V_{i,old}(h_{i,t}) - \epsilon, V_{i,old}(h_{i,t}) + \epsilon) - \hat{R}_t \right)^2 \right] \quad (4.17)$$

¹⁰Here, I consider the stochastic (not continuous) action case and for simplicity do not include GAE, mini-batching, or a policy entropy term (which is common in policy gradient methods to improve exploration). Also, the paper uses a local advantage loss, A_i , but this value would not be different than $\mathbf{A}$ in a general Dec-POMDP if it is updated using $\mathbf{A}$ and r since these quantities would be the same for all agents. Finally, policies over single observations rather than histories are used, which as noted before is typically not sufficient for partially observable problems (general Dec-POMDPs) and state-based critics are used, which is incorrect as discussed below.

where $\hat{R}$ are Monte Carlo returns starting from the current *local* history, $V_i(h_{i,t})$. Parameter sharing is used with IPPO so all agents share the actor and critic network, making it a CTDE approach. As a result, there is only one actor and one critic, but it is updated using the data from all agents.

MAPPO and IPPO perform very well on the standard benchmark domains. Both algorithms typically perform similarly, but a version of MAPPO where the information in the critic was hand designed to only include features that are relevant to the task could outperform the standard versions of MAPPO and IPPO. This makes sense as the agents did not have to learn what information was necessary, which is difficult in high-dimensional settings.

4.3.6 State-based critics

Critics that use state (and not history) are unsound in partially observable environments (i.e., general Dec-POMDPs). That is, they are incorrect and could be arbitrarily bad. They can work well in domains that are nearly fully observable (such as SMAC [Samvelyan et al., 2019]) and domains in which remembering history information is not helpful (observations are not very informative or the task is too hard to solve). This result has been shown theoretically and empirically [Lyu et al., 2022, 2023].

In particular, state-based critics estimate $Q(s, a)$ or $V(s)$. MADDPG [Lowe et al., 2017] and COMA [Foerster et al., 2018b] popularized the idea of using state-based critics using the intuition that this ground truth information that is available during training can help address partial observability. In truth, it is typically a bad idea to eliminate partial observability in the critic. A Dec-POMDP typically *is* partially observable and the actor is using partially observable information (i.e., histories). The state-based critic can be biased (i.e., incorrect) because there is a mismatch between the actor and the critic. The actor needs to use history information to choose actions. The state-based critic uses state values instead. As a result, the state values are averaged over the histories that are visited and can't possibly have the correct history values unless there is a one-to-one mapping from states to histories (which would happen in the fully observable case). That is, the critic loses the information about partial observability, which is necessary for the actor to make good choices.

For example, in the class Dec-Tiger problem [Nair et al., 2003, Oliehoek and Amato, 2016], there are only 2 states (the tiger being on the left or on the right) but noisy observations are received when an agent listens (an agent can also choose to open one of the two doors). A state-based critic would only have two values $V(s^1)$ and $V(s^2)$ but there are an exponential number of histories that depends on the horizon. As more information is gathered (i.e., the same observation about the tiger's location is heard multiple times) the value of the history, $V(h)$, should increase (so $V(h_{\text{tiger-left-10-times-in-a-row}}) > V(h_0)$, the initial history). This can't be reflected in the state-based critic. Furthermore, information-gathering actions (i.e., listening in this tiger problem) won't improve the value according to the critic so they shouldn't be taken. As a result, the state-based critic values will be incorrect in the Dec-Tiger problem and the agents would learn a policy to open one of the doors and hope for a good outcome (assuming the tiger is randomly initialized).

The updates to Algorithm 9 for the state-based critic case are straightforward— $\hat{V}(s)$ is used in place of $\hat{V}(h)$. Then, the TD error calculation becomes $\delta_t \leftarrow r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t)$, which is used in the actor and critic updates.

It turns out that history-state critics, those that take both the state and history as input ($Q(h, s, a)$ or $V(h, s)$) are unbiased and often perform the best [Lyu et al., 2022, 2023, Yu et al., 2022]. This

result has been theoretically and empirically shown for the general single and multi-agent cases [Baisero and Amato, 2022, Lyu et al., 2022, 2023] as well as empirically with variants of MAPPO [Yu et al., 2022]. The intuition for correctness of the history-state critic is that no history information is lost (as it is in the simpler state-based critic). That is, the correct history value function can be (and is) recovered from the history-state value: $Q^\pi(\mathbf{h}, \mathbf{a}) = \mathbb{E}_{s|\mathbf{h}}[Q^\pi(\mathbf{h}, s, \mathbf{a})]$. The MAPPO paper also showed that if you can handcraft the features of the history-state critic to retain only the relevant information, the performance can be the highest since the agent doesn't need to learn which information is relevant and which is not. Unfortunately, determining which information is relevant and properly separating it can be difficult to do. The algorithmic updates for the history-state critic case are also straightforward— $\hat{V}(\mathbf{h}, s)$ is used in place of $\hat{V}(\mathbf{h})$. The TD error calculation becomes $\delta_t \leftarrow r_t + \gamma \hat{V}(\mathbf{h}, s_{t+1}) - \hat{V}(\mathbf{h}, s_t)$, which is used in the actor and critic updates.

4.3.7 Choosing different types of decentralized and centralized critics

As noted in Section 3.4, while CTDE methods are popular, they don't always perform the best and are often quite close to DTE methods. This is because DTE methods typically make the concurrent learning assumption where all the agents learn using the same algorithm, making updates at the same time on the joint data. As mentioned, while centralized critic actor-critic methods are often assumed to be better than DTE actor-critic methods, they are theoretically the same (with mild assumptions) and often empirically similar (and sometimes worse) [Peshkin et al., 2000, Yu et al., 2022, Lyu et al., 2023].

The theory assumes learned critics (or sufficient Monte Carlo estimates) but different types of critics may be easier to learn in different cases. As a result, it is not straightforward to determine when to use each critic in practice. Centralized critics often cause higher variance actor updates since information from the other agents needs to be marginalized out (e.g., through sampling) while information from the other agents is already removed from the decentralized critics. Centralized critics can also be more difficult to learn in problems with large numbers of agents or with large action or observation sets, making decentralized critics more scalable. Decentralized critics may be difficult to learn due to agents changing their policies (i.e., nonstationarity), making critic estimates stale. State-based critics are often the easiest to learn (because no history representation needs to be learned) but are biased in partially observable settings and will have high variance (again, because of the mismatch between the actor and the critic). In general, learning a history representation is difficult. Agents need to figure out what history information is relevant and what is not from a noisy and often sparse RL signal. Using a recurrent network (or even a transformer) is usually not sufficient for this task. History-state critics further increase the variance in the policy update (because the state information needs to also be marginalized out) but can often be easier to learn than centralized critics with only history information.¹¹ As a result, the choice of critic is often a bias-variance tradeoff since, in practice, a centralized critic should have a lower bias than decentralized critics with more stable values that are more easily updated when policies change but higher variance because the policy updates need to be averaged over other agents.

¹¹The exact reason why this is the case is unclear and a great topic for future research!

4.3.8 Methods that combine policy gradient and value factorization

Several methods combine using a centralized critic with value factorization. One notable method is FACTored Multi-Agent Centralized policy gradients (FACMAC) [Peng et al., 2021] which extends MADDPG to include a QMIX-style factored centralized critic, but since the local Q-values don't need to be used for action selection (the actor does that), the factorization can be nonmonotonic. Also, rather than sampling the actions of other agents from the off-policy dataset, as MADDPG does, actions for other agents are sampled from the current policies during the actor updates. Decomposed Off-Policy policy gradient (DOP) [Wang et al., 2021c] uses a factored centralized critic that is a weighted sum of the local Q-values and a multi-agent extension of tree backup [Precup et al., 2000] to more efficiently calculate off-policy updates. While a centralized critic is learned, each agent's local Q-value estimate is used to update its actor and with mild assumptions the method is shown to converge to a local optimum even with the simple critic.

4.3.9 Other centralized critic methods

Many other methods use centralized critics. For instance, Iqbal and Sha [2019] use attention for determining which other agent information to include in the centralized critic. Other approaches are related to the ones presented here but some form of MAPPO is still the most widely used centralized critic method.

4.4 Other forms of CTDE

While the methods above are the ones that are most widely used, there are several other forms of CTDE. I discuss common approaches and include a note on the many other topics that are not in this text in order to focus on core issues of CTDE.

4.4.1 Adding centralized information to decentralized methods

Decentralized training and execution (DTE) methods can be augmented with centralized information to potentially improve performance. Such methods are only possible when a centralized training phase is available for sharing information but the resulting methods seek to balance scalability and performance. These extensions are discussed below.

Parameter sharing Many cooperative MARL methods use parameter sharing. As noted in Section 4.3.5, the idea behind parameter (or weight) sharing is instead of each agent using a different network to estimate the value function or policy, all agents share the same networks. More details about parameter sharing are given in Section 3.4. Since parameter sharing requires agents to share networks during training, I consider it a form of CTDE.

Alternating learning A number of methods allow agents to take turns learning. For instance, DTE methods can be used but all agents are fixed (not learning) except for one. The learning agent learns until convergence, generating a best-response to the other agent policies. If this process continues until no agents can further improve their policies, the result is a Nash equilibrium [Banerjee

et al., 2012, Su et al., 2024]. In limited settings with additional strong assumptions (e.g., deterministic environments, full observability, additional coordination mechanisms to ensure coordinated policies), such methods can potentially converge to optimal solutions [Lauer and Riedmiller, 2000, Jiang and Lu, 2023, 2022]. While these methods are sometimes called ‘decentralized’ since coordination and communication is needed during learning that is not available during execution, I consider them to be CTDE.

Kuba et al. [2022] use sequential agent updates (i.e., one agent updates at a time while holding the others fixed) in Heterogeneous-Agent Trust Region Policy Optimisation (HATRPO) and Heterogeneous-Agent Proximal Policy Optimisation (HAPPO). HATRPO and HAPPO build off of TRPO [Schulman et al., 2015] and PPO [Schulman et al., 2017] and remove the parameter sharing used in MAPPO [Yu et al., 2022] (leading to the heterogeneous-agent name). Using this sequential update scheme, they can theoretically prove monotonic improvement for the fully observable case and the approaches can work well in practice.

Addressing nonstationarity Decentralized learning methods face a nonstationary problem due to changing policies of other agents. Some methods try to address this challenge by directly modeling these changes. For example, Foerster et al. [2017] propose using importance sampling to correct for the probability differences and decay old data or simpler ‘fingerprints’ (e.g., episode number) to mark the data’s age. Other methods try to model the other agent learning updates as well [Foerster et al., 2018a, Willi et al., 2022].

4.4.2 Decentralizing centralized solutions

Another potential CTDE approach is to learn a centralized solution during training and then (attempt to) decentralize it before execution. This is much harder than it seems. A centralized policy can map $\mathbb{H} \rightarrow \mathbb{A}$ without the constraint (in the stochastic case) that $\pi(\mathbf{a}|\mathbf{h}) = \prod_{i \in \mathbb{I}} \pi_i(a_i|h_i)$. That is, each agent’s policy can depend on *other agent histories*: $\pi(\mathbf{a}|\mathbf{h}) = \prod_{i \in \mathbb{I}} \pi_i(a_i|\mathbf{h})$. The centralized policy class is much larger and richer than the decentralized policy class.

For example, just considering deterministic policies of horizon $\mathcal{H}$, there would be $|\mathbb{A}_i|^{|\mathbb{Q}_i|^{\mathcal{H}|\mathbb{I}}}$ possible decentralized policies (assuming all agents have the same size action and observation sets) vs. $|\mathbb{A}|^{|\mathbb{Q}^{\mathcal{H}}|}$ possible centralized policies. Plugging in 4 actions per agent, 5 observations, a horizon (history-length) of 10 and 4 agents gives $4^{5^{10^4}} = 1.024 \times 10^{43}$ vs. $(4^4)^{(5^4)^{10}} = 2.56 \times 10^{62520}$ policies. This is a massive difference.

As a result, most centralized policies will not be directly decentralizable in the sense that they will be equivalent.¹² Policies can only be decentralized when they don’t depend on other agent information but centralized agents that make joint action choices based on the information of all agents can often perform much better. In fact, this centralized partially observable case is called a multi-agent POMDP and can be solved using single-agent methods as discussed in Chapter 2.

As a result of these issues, approaches that decentralize centralized solutions are not common. Nonetheless, there are some instances. For example, Li et al. [2023] attempt to reconstruct the global information using a reconstruction loss, other methods attempt to mimic a centralized controller [Lin et al., 2022], and others allow communication during training but reduce or remove it during execution [Foerster et al., 2016, Wang et al., 2020]. FOP (for Factorizes the Optimal joint

¹²Note that this is not true in the fully observable case as discussed in Section 1.2.

Policy) [Zhang et al., 2021] makes the assumption that centralized solutions are decentralizeable and then seeks to learn solutions using a max entropy formulation along with a value factorization scheme that is similar to methods in Section 4.2. While FOP (and other methods) could be applied in general Dec-POMDP settings, they are likely to perform poorly when the centralized solution is sufficiently different from the decentralized solution.

Chapter 5

Conclusion

5.1 Topics not discussed

There are many other issues that are important to MARL but are not discussed in this text, including exploration and communication as well as other topics such as offline methods, model-based methods, hierarchical methods, role decomposition, ad-hoc (or zero-shot) coordination, and multi-task approaches. These topics, while important, are excluded here to maintain brevity and focus. Some surveys of these areas include communication [Zhu et al., 2024], ad hoc teamwork [Mirsky et al., 2022], and model-based methods [Wang et al., 2022].

5.2 Evaluation domains

While earlier work focused on domains such as multi-agent particle environments [Lowe et al., 2017] and the Starcraft Multi-Agent Challenge (SMAC) [Samvelyan et al., 2019], recently, many more domains have become available. Some examples include a more complex version of SMAC that is no longer deterministic and includes more partial observability [Ellis et al., 2024] as well as soccer-based environments [Kurach et al., 2020]. Resources like pettingzoo¹ provide environment code, while algorithm implementations are available through platforms such as BenchMARL (based in TorchRL) [Bettini et al., 2024] and JAXMARL (based in JAX) [Rutherford et al., 2024]. More details can also be found through the various papers (which typically include links to code) and the book by Albrecht et al. [2024].

5.3 Applications

Cooperative MARL is increasingly applied across diverse domains, including gaming, robotics, and traffic systems. Some examples include AlphaStar for playing StarCraft [Vinyals et al., 2019], multi-robot systems such as in warehouses [Krnjaic et al., 2024, Xiao et al., 2025], military drones [Kim et al., 2024], and multi-robot pathfinding [Damani et al., 2021] as well as traffic signal control [Van der Pol and Oliehoek, 2016, Wei et al., 2021], autonomous driving [Zhang et al., 2024], and power systems [Wang et al., 2021b]. Many more examples exist and I expect many more will exist in the near future.

¹<https://pettingzoo.farama.org/>

5.4 Future topics

There are many different ways of performing cooperative MARL and current work has only scratched the surface. Studying the differences and similarities between DTE and CTDE variants of algorithms seems like a promising direction for understanding current methods and developing improved approaches for both cases. Determining what centralized information to use and how to best use it is another key question. Lastly, to the best of my knowledge, there are no globally optimal model-free MARL methods for Dec-POMDPs. This is surprising since there are optimal *planning* methods where the model is assumed known [Oliehoek and Amato, 2016] and many globally optimal model-free methods for single-agent RL [Sutton and Barto, 2018]. Developing such methods (even for the tabular case) would be interesting and may lead to new, better methods that could be approximated in the deep case.

Acknowledgements

I thank Andrea Baisero, Shuo Liu, Roi Yehoshua, and the other members of my Lab for Learning and Planning in Robotics (LLRP) for reading my (very) rough drafts and providing comments that helped improve the document as well as Frans Oliehoek for the many discussions about these topics and the resulting text. I also thank NSF CAREER Award 2044993 for funding this effort.

Bibliography

- S. V. Albrecht, F. Christianos, and L. Schäfer. *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. <https://www.marl-book.com>.
- C. Amato and F. A. Oliehoek. Scalable planning and learning for multiagent POMDPs. In *Proceedings of the National Conference on Artificial Intelligence*, pages 1995–2002, Jan. 2015.
- A. Baisero and C. Amato. Unbiased asymmetric reinforcement learning under partial observability. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2022.
- B. Banerjee, J. Lyle, L. Kraemer, and R. Yellamraju. Sample bounded distributed reinforcement learning for decentralized POMDPs. In *Proceedings of the National Conference on Artificial Intelligence*, 2012.
- M. G. Bellemare, W. Dabney, and M. Rowland. *Distributional Reinforcement Learning*. MIT Press, 2023. <http://www.distributional-rl.org>.
- D. S. Bernstein, R. Givan, N. Immerman, and S. Zilberstein. The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research*, 27(4):819–840, 2002.
- M. Bettini, A. Prorok, and V. Moens. Benchmarl: Benchmarking multi-agent reinforcement learning. *Journal of Machine Learning Research*, 25(217):1–10, 2024.
- W. Böhmer, V. Kurin, and S. Whiteson. Deep coordination graphs. In *Proceedings of the International Conference on Machine Learning*, 2020.
- C. Boutilier. Planning, learning and coordination in multiagent decision processes. In *Proceedings of the 6th Conference on Theoretical Aspects of Rationality and Knowledge*, 1996.
- L. Buşoniu, R. Babuška, and B. De Schutter. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, 38(2):156–172, Mar. 2008.
- C. Claus and C. Boutilier. The dynamics of reinforcement learning in cooperative multiagent systems. In *Proceedings of the National Conference on Artificial Intelligence*, pages 746–752, 1998.
- M. Damani, Z. Luo, E. Wenzel, and G. Sartoretti. Primal₂: Pathfinding via reinforcement and imitation multi-agent learning-lifelong. *IEEE Robotics and Automation Letters*, 6(2):2666–2673, 2021.

- C. S. de Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. Torr, M. Sun, and S. Whiteson. Is independent learning all you need in the StarCraft multi-agent challenge? *arXiv preprint arXiv:2011.09533*, 2020.
- B. Ellis, J. Cook, S. Moalla, M. Samvelyan, M. Sun, A. Mahajan, J. Foerster, and S. Whiteson. SMACv2: An improved benchmark for cooperative multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2024.
- K. Esslinger, R. Platt, and C. Amato. Deep transformer Q-networks for partially observable reinforcement learning. *arXiv preprint arXiv:2206.01078*, 2022.
- J. Foerster, I. A. Assael, N. De Freitas, and S. Whiteson. Learning to communicate with deep multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 29, 2016.
- J. Foerster, N. Nardelli, G. Farquhar, T. Afouras, P. H. Torr, P. Kohli, and S. Whiteson. Stabilising experience replay for deep multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, pages 1146–1155, 2017.
- J. Foerster, R. Y. Chen, M. Al-Shedivat, S. Whiteson, P. Abbeel, and I. Mordatch. Learning with opponent-learning awareness. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2018a.
- J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the National Conference on Artificial Intelligence*, 2018b.
- C. V. Goldman and S. Zilberstein. Decentralized control of cooperative systems: Categorization and complexity analysis. *Journal of AI Research*, 22:143–174, 2004.
- C. Guestrin, D. Koller, and R. Parr. Multiagent Planning with Factored MDPs. In *Advances in Neural Information Processing Systems*, 2001.
- C. Guestrin, M. Lagoudakis, and R. Parr. Coordinated reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, pages 227–234, 2002.
- J. K. Gupta, M. Egorov, and M. Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In *Adaptive and Learning Agents Workshop at AAMAS*, 2017.
- D. Ha, A. M. Dai, and Q. V. Le. Hypernetworks. In *Proceedings of the International Conference on Learning Representations*, 2017.
- T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the International Conference on Machine Learning*, 2018.
- M. Hausknecht and P. Stone. Deep recurrent Q-learning for partially observable MDPs. *arXiv preprint arXiv:1507.06527*, 2015.
- Y.-C. Ho. Team decision theory and information structures. *Proceedings of the IEEE*, 68(6): 644–654, 1980.

- S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- A. A. Hussain, F. Belardinelli, and G. Piliouras. Asymptotic convergence and performance of multi-agent Q-learning dynamics. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2023.
- S. Iqbal and F. Sha. Actor-attention-critic for multi-agent reinforcement learning. In *International conference on machine learning*, 2019.
- T. Jaakkola, M. Jordan, and S. Singh. Convergence of stochastic iterative dynamic programming algorithms. In *Advances in Neural Information Processing Systems*, volume 6, 1993.
- J. Jiang and Z. Lu. I2Q: A fully decentralized Q-learning algorithm. In *Advances in Neural Information Processing Systems*, 2022.
- J. Jiang and Z. Lu. Best possible Q-learning. *arXiv preprint arXiv:2302.01188*, 2023.
- L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- G. S. Kim, S. Lee, T. Woo, and S. Park. Cooperative reinforcement learning for military drones over large-scale battlefields. *IEEE Transactions on Intelligent Vehicles*, pages 1–11, 2024.
- J. R. Kok and N. Vlassis. Collaborative Multiagent Reinforcement Learning by Payoff Propagation. *Journal of Machine Learning Research*, 7:1789–1828, Dec. 2006a. ISSN 1532-4435.
- J. R. Kok and N. Vlassis. Collaborative multiagent reinforcement learning by payoff propagation. *Journal of Machine Learning Research*, 7:1789–1828, 2006b.
- L. Kraemer and B. Banerjee. Multi-agent reinforcement learning as a rehearsal for decentralized planning. *Neurocomputing*, 190:82–94, 2016.
- A. Krnjaic, R. D. Stealec, J. D. Thomas, G. Papoudakis, L. Schäfer, A. W. K. To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Börsting, and S. V. Albrecht. Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers. In *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024.
- J. G. Kuba, R. Chen, M. Wen, Y. Wen, F. Sun, J. Wang, and Y. Yang. Trust region policy optimisation in multi-agent reinforcement learning. In *Proceedings of the International Conference on Learning Representations*, 2022.
- K. Kurach, A. Raichuk, P. Stańczyk, M. Zajac, O. Bachem, L. Espeholt, C. Riquelme, D. Vincent, M. Michalski, O. Bousquet, et al. Google research football: A novel reinforcement learning environment. In *Proceedings of the National Conference on Artificial Intelligence*, 2020.
- M. Lauer and M. A. Riedmiller. An algorithm for distributed reinforcement learning in cooperative multi-agent systems. In *Proceedings of the International Conference on Machine Learning*, 2000.

- S. Levine, A. Kumar, G. Tucker, and J. Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- D. Li, Z. Xu, B. Zhang, G. Zhou, Z. Zhang, and G. Fan. From explicit communication to tacit cooperation: A novel paradigm for cooperative marl. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2023.
- S. Li, J. K. Gupta, P. Morales, R. Allen, and M. J. Kochenderfer. Deep implicit coordination graphs for multi-agent reinforcement learning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2021.
- T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. In *Proceedings of the International Conference on Learning Representations*, 2016.
- A. T. Lin, M. DeBord, K. Estabridis, G. Hower, G. Montufar, and S. Osher. Decentralized multi-agents by imitation of a centralized controller. In *Mathematical and Scientific Machine Learning*, pages 619–651, 2022.
- L.-J. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*, 8(3-4):293–321, 1992.
- M. L. Littman. Value-function reinforcement learning in markov games. *Cognitive systems research*, 2(1):55–66, 2001.
- M. L. Littman, T. L. Dean, and L. P. Kaelbling. On the complexity of solving Markov decision problems. In *Proceedings of Uncertainty in Artificial Intelligence*, 1995.
- R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.
- X. Lyu and C. Amato. Likelihood quantile networks for coordinating multi-agent reinforcement learning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2020.
- X. Lyu, Y. Xiao, B. Daley, and C. Amato. Contrasting centralized and decentralized critics in multi-agent reinforcement learning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2021.
- X. Lyu, Y. Xiao, A. Baisero, and C. Amato. A deeper understanding of state-based critics in multi-agent reinforcement learning. In *Proceedings of the National Conference on Artificial Intelligence*, 2022.
- X. Lyu, A. Baisero, Y. Xiao, B. Daley, and C. Amato. On centralized critics in multi-agent reinforcement learning. *Journal of AI Research*, 77:235–294, 2023.
- X. Lyu, A. Baisero, Y. Xiao, B. Daley, and C. Amato. On centralized critics in multi-agent reinforcement learning (updated version). *arXiv preprint arXiv: 2408.14597*, 2024.

- E. Marchesini, A. Baisero, R. Bathi, and C. Amato. On stateful value factorization in multi-agent reinforcement learning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2025.
- J. Marschak. Elements for a theory of teams. *Management Science*, 1:127–137, 1955.
- L. Matignon, G. J. Laurent, and N. Le Fort-Piat. Hysteretic Q-learning: An algorithm for decentralized reinforcement learning in cooperative multi-agent teams. In *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 64–69, 2007.
- F. S. Melo. Convergence of Q-learning: A simple proof. *Institute Of Systems and Robotics, Tech. Rep*, 2001.
- R. Mirsky, I. Carlucho, A. Rahman, E. Fosong, W. Macke, M. Sridharan, P. Stone, and S. V. Albrecht. A survey of ad hoc teamwork research. In *European Conference on Multi-Agent Systems*, 2022.
- V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529, 2015.
- T. Morimura, M. Sugiyama, H. Kashima, H. Hachiya, and T. Tanaka. Nonparametric return distribution approximation for reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, pages 799–806, 2010.
- K. P. Murphy. A survey of POMDP solution techniques. Technical report, University of British Columbia, 2000.
- R. Nair, M. Tambe, M. Yokoo, D. V. Pynadath, and S. Marsella. Taming decentralized POMDPs: Towards efficient policy computation for multiagent settings. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pages 705–711, 2003.
- K. X. Nguyen. Converting POMDPs into MDPs using history representation. *engrxiv preprint*, 2021.
- T. Ni, M. Ma, B. Eysenbach, and P.-L. Bacon. When do transformers shine in RL? Decoupling memory from credit assignment. In *Advances in Neural Information Processing Systems*, 2023.
- C. Nota and P. S. Thomas. Is the policy gradient a gradient? In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2020.
- F. A. Oliehoek and C. Amato. *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.
- S. Omidshafiei, J. Pazis, C. Amato, J. P. How, and J. Vian. Deep decentralized multi-task multi-agent reinforcement learning under partial observability. In *Proceedings of the International Conference on Machine Learning*, 2017.
- G. Palmer, K. Tuyls, D. Bloembergen, and R. Savani. Lenient multi-agent deep reinforcement learning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, pages 443–451, 2018.

- C. H. Papadimitriou and J. N. Tsitsiklis. The complexity of Markov decision processes. *Mathematics of operations research*, 12(3):441–450, 1987.
- B. Peng, T. Rashid, C. Schroeder de Witt, P.-A. Kamienny, P. Torr, W. Böhmer, and S. Whiteson. Facmac: Factored multi-agent centralised policy gradients. *Advances in Neural Information Processing Systems*, 2021.
- L. Peshkin, K.-E. Kim, N. Meuleau, and L. P. Kaelbling. Learning to cooperate via policy search. In *Proceedings of Uncertainty in Artificial Intelligence*, 2000.
- D. Precup, R. S. Sutton, and S. P. Singh. Eligibility traces for off-policy policy evaluation. In *Proceedings of the International Conference on Machine Learning*, 2000.
- M. L. Puterman. *Markov Decision Processes—Discrete Stochastic Dynamic Programming*. John Wiley & Sons, Inc., 1994.
- R. Radner. Team decision problems. *Annals of Mathematical Statistics*, 33:857–881, 1962.
- T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2018.
- T. Rashid, G. Farquhar, B. Peng, and S. Whiteson. Weighted QMIX: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020a.
- T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(1):7234–7284, 2020b.
- A. Rutherford, B. Ellis, M. Gallici, J. Cook, A. Lupu, G. Ingvarsson, T. Willi, A. Khan, C. Schroeder de Witt, A. Souly, et al. JaxMARL: Multi-agent RL environments and algorithms in JAX. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2024.
- M. Samvelyan, T. Rashid, C. S. de Witt, G. Farquhar, N. Nardelli, T. G. J. Rudner, C.-M. Hung, P. H. S. Torr, J. Foerster, and S. Whiteson. The StarCraft Multi-Agent Challenge. *arXiv preprint arXiv:1902.04043*, 2019.
- J. Schneider, W.-K. Wong, A. Moore, and M. Riedmiller. Distributed value functions. In *Proceedings of the International Conference on Machine Learning*, 1999.
- J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *Proceedings of the International Conference on Machine Learning*, 2015.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- S. Sen, M. Sekaran, J. Hale, et al. Learning to coordinate without sharing information. In *Proceedings of the National Conference on Artificial Intelligence*, volume 94, pages 426–431, 1994.

- D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *Proceedings of the International Conference on Machine Learning*, 2014.
- K. Son, D. Kim, W. J. Kang, D. E. Hostallero, and Y. Yi. QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2019.
- K. Su, S. Zhou, J. Jiang, C. Gan, X. Wang, and Z. Lu. MA2QL: A minimalist approach to fully decentralized multi-agent reinforcement learning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2024.
- P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel. Value-decomposition networks for cooperative multi-agent learning. *arXiv:1706.05296*, 2017.
- R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction (second edition)*. The MIT Press, 2018.
- A. Tampuu, T. Matiisen, D. Kodelja, I. Kuzovkin, K. Korjus, J. Aru, J. Aru, and R. Vicente. Multiagent cooperation and competition with deep reinforcement learning. *PloS one*, 12(4), 2017.
- M. Tan. Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the International Conference on Machine Learning*, pages 487–494, 1993.
- J. K. Terry, N. Grammel, S. Son, B. Black, and A. Agrawal. Revisiting parameter sharing in multi-agent deep reinforcement learning. *arXiv preprint arXiv:2005.13625*, 2020.
- K. Tuyls, K. Verbeeck, and T. Lenaerts. A selection-mutation model for Q-learning in multi-agent systems. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, pages 693–700, 2003.
- E. Van der Pol and F. A. Oliehoek. Coordinated deep reinforcement learners for traffic light control. *Proceedings of Workshop on Learning, Inference and Control of Multi-Agent Systems at NeurIPS*, 2016.
- O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- J. Wang, Z. Ren, T. Liu, Y. Yu, and C. Zhang. QPLEX: Duplex dueling multi-agent Q-learning. In *Proceedings of the International Conference on Learning Representations*, 2021a.

- J. Wang, W. Xu, Y. Gu, W. Song, and T. C. Green. Multi-agent reinforcement learning for active voltage control on power distribution networks. In *Advances in Neural Information Processing Systems*, 2021b.
- T. Wang, J. Wang, C. Zheng, and C. Zhang. Learning nearly decomposable value functions via communication minimization. In *Proceedings of the International Conference on Learning Representations*, 2020.
- X. Wang, Z. Zhang, and W. Zhang. Model-based multi-agent reinforcement learning: Recent progress and prospects. *arXiv preprint arXiv:2203.10603*, 2022.
- Y. Wang, B. Han, T. Wang, H. Dong, and C. Zhang. DOP: Off-policy multi-agent decomposed policy gradients. In *Proceedings of the International Conference on Learning Representations*, 2021c.
- Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2016.
- C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8(3):279–292, May 1992.
- E. Wei and S. Luke. Lenient learning in independent-learner stochastic cooperative games. *The Journal of Machine Learning Research*, 17(1):2914–2955, 2016.
- H. Wei, G. Zheng, V. Gayah, and Z. Li. Recent advances in reinforcement learning for traffic signal control: A survey of models and evaluation. *ACM SIGKDD Explorations Newsletter*, 22(2):12–18, 2021.
- T. Willi, A. H. Letcher, J. Treutlein, and J. Foerster. COLA: Consistent learning with opponent-learning awareness. In *Proceedings of the International Conference on Machine Learning*, 2022.
- R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.
- M. Wunder, M. L. Littman, and M. Babes. Classes of multiagent Q-learning dynamics with ϵ -greedy exploration. In *Proceedings of the International Conference on Machine Learning*, pages 1167–1174, 2010.
- Y. Xiao, W. Tan, J. Hoffman, T. Xia, and C. Amato. Asynchronous multi-agent deep reinforcement learning under partial observability. *The International Journal of Robotics Research*, 2025.
- Y. Yang, J. Hao, B. Liao, K. Shao, G. Chen, W. Liu, and H. Tang. Qatten: A general framework for cooperative multiagent reinforcement learning. *arXiv preprint arXiv:2002.03939*, 2020.
- C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen, and Y. Wu. The surprising effectiveness of PPO in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 35, 2022.
- K. Zhang, Z. Yang, H. Liu, T. Zhang, and T. Basar. Fully decentralized multi-agent reinforcement learning with networked agents. In *Proceedings of the International Conference on Machine Learning*, pages 5872–5881, 10–15 Jul 2018.

- R. Zhang, J. Hou, F. Walter, S. Gu, J. Guan, F. Röhrbein, Y. Du, P. Cai, G. Chen, and A. Knoll. Multi-agent reinforcement learning for autonomous driving: A survey. *arXiv preprint arXiv:2408.09675*, 2024.
- T. Zhang, Y. Li, C. Wang, G. Xie, and Z. Lu. FOP: Factorizing optimal joint policy of maximum-entropy multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2021.
- C. Zhu, M. Dastani, and S. Wang. A survey of multi-agent deep reinforcement learning with communication. *Journal of Autonomous Agents and Multi-Agent Systems*, 38(1):4, 2024.